TM

The Premise Guide

version 3.3

by

Michael S P Miller

The Premise Guide

The Premise Guide

version 3.3

by

Michael S. P. Miller

For comments on the SubThought Premise Language contact: premise.ai@gmail.com

For electronic inquiries or permissions contact: subthought@hotmail.com.

by mail: SubThought Corporation, 311 N. Robertson Blvd #301, Beverly Hills, CA 90211 USA

This book was published in the United States of America.

Paperback ISBN-13: 979-8-849321-64-6
Paperback ISBN-10: 8-849321-64-3
Hardcover ISBN-13: 979-8-849796-21-5
Hardcover ISBN-10: 8-849796-21-830

Cover by Michael S. P. Miller
Document version: 3.336

For those who seek Truth.

Contents

Figures

Tables

Acknowledgements

"Do you see over yonder, friend Sancho, thirty or forty hulking giants? I intend to do battle with them and slay them."

— Miguel de Cervantes Saavedra, Don Quixote

This work was not possible without the efforts and advice of Dr. Sheldon Linker, whose collaboration on terminology and functionality was most welcome, and combined with his proficient coding skills led to the 2013 Java implementation of a just in time interpreter: The Premise 0.1 Executive. Suzuki Hisao's lucid and concise C# implementation of Nukata Lisp Lite was insightful, as was Peter Norvig's implementation of JScheme. Allen Holub's book Compiler Design in C was also instructive, particularly in the area regarding approaches to tokenization and parsing. Christian Queinnec's tome, Lisp In Small Pieces, also provided invaluable insights on expression evaluation. Christian's encouragement on this project was greatly appreciated. Roland Hausser's left associative grammar was pivotal in the tokenizer design.

The author would also like to thank Aaron Hosford for his suggestions on numeric similarity metrics, Ryan Hewitt for his comments on prototype construction, Brian Will for his ideas on restricted scoping, Dr. Ghassan Azar, Todd Kaufmann, Dr. Michael Schuresko, Robert Rossi, Jean-Louis Villecroze and the rest of the Premise Language group for their constructive feedback and suggestions.

Disclaimer

Introduction

The Premise Language is a functional prototype programming language which combines high level intrinsic primitives with seamless object persistence. The goal of the Premise Language is to provide a platform for artificial intelligence programming. Software agents written in Premise share a knowledge base where they can create, modify, and delete object instances amongst themselves in a stigmergic manner. Premise is influenced by Lisp, self, JavaScript, OPS5, and CLIPS.

The Premise Language was developed by Michael S. P. Miller and Sheldon O. Linker during 2013 and 2014. In creating a platform for agent based programming for intelligent systems (JCB English and The Piagetian Modeler) they found a need for a very high level language in which to code asynchronous agent processes to perform complex pattern matching and inference. They explored the Sapir-Whorf hypothesis as it relates to computer programming–namely, that the primitive operations available in a computer language influence the way software developers frame and solve problems, and it was early determined that the primitives of the needed language should be very high level and include logical and similarity-based pattern matching, inference, messaging, stigmergy, and asynchrony. The language combines elements of declarative, functional, and imperative programming with seamless persistent object storage and retrieval. The goals of The Premise Language are simple:

- To define memory as a portable abstraction across different physical implementations. Premise uses a persistent object store as its memory.
- To facilitate the fast formation of relationships in a persistent memory. Constructing persisted records in Premise is the same as asserting facts to the working memory of other declarative language platforms such as CLIPS or OPS.
- To allow rapid interrogation of relationships in the memory.
- To provide scalability to billions of persisted records.
- To explore the Sapir Whorf hypothesis as it relates to computer programming: that the constructs available in computer languages influence the way people think about and solve problems.
- To explore the stigmergic programming paradigm, where objects constructed by one agent are later used or consumed by other agents.
- To provide a clear and concise API.

The Premise Language serves as a testbed for agent programming. Functions are the primary unit of processing. Numbers, strings, literals, prototypes, persisted records, lexicons, lists and calls provide the primary data types. The Premise Language opens up new possibilities for knowledge representation and artificial intelligence programming.

We hope you enjoy using this language.

Michael Miller

February 2019 (version 1.0)
July 2025 (version 3.0)

Document Conventions

The following are conventions used in this document.

Source Code

Premise source code will appear in Courier New font in a colored text region. The gray text region depicts a session in the Premise interpreter.

```
> (parse "{a b c}")

.: {a b c}  8
```

A green text region depicts Premise source code snippets that appear in a file editor.

```
(relation Problem
  :Name  :Status
)
```

A blue text region depicts non-Premise source code snippets that appear in a file editor.

```
SELECT DISTINCT LENGTH(CategoryName)
  FROM Category
```

Source Code Comments

In-line comments are denoted by semicolons. Everything from the comment to the end of the line is ignored by the interpreter.

```
   ; this is an in-line comment
```

Multi-line comments are delimited by double quotation marks. Everything between the delimiters is treated as a single string and returned as-is by the interpreter. A free standing string, not part of a function or macro call, evaluates to itself, and thus can be used as a comment.

```
   "this is a
   multi-line
   comment"
```

Language Style Conventions

As a matter of style, each of the following language elements is written in a specific case:

Modules	are written in Pascal case.
Types	are written in Pascal case.
:Slots	are written in Pascal case with a preceding colon.
!methods	are written in camel case with a preceding exclamation point.
functions	use camel case for verbs and Pascal case for nouns.
?locals	(local variables) are in camel case with a preceding question mark.
?Module.Name	(modular variables) are in Pascal case with a preceding question mark.
?Domain.Name	(domain variables) are in Pascal case with a preceding question mark.
?*Global*	(global variables) are in Pascal case with a preceding question mark.
?CONSTANTS	are written in upper case with a preceding question mark.

1. Getting Started

To use the Premise Language, download the Premise executable to a folder on your computer then extract the file using standard decompression software.

To run the Premise evaluator type the following at your operating system command line:

premise

Example:

```
C:\projects\SubThought\> premise

 [Premise :Version 3.3 :Build 20260301.0001 :OS Windows 11 :Edition Community]

 Type expressions then press the [Enter] key twice. For information type (help),
 (about), (license), or (copyright). To read a file type (grok "path/file").
 Type (bye) to exit.

 >  (copyright)

 .: "Copyright (c) 2014-2026 SubThought Corporation, All Rights Reserved."

 >
```

The command line options for the Premise interpreter are as follows.

--home *url*	sets the session start folder .
--repl *yes* | *no*	If *no* then processes the command line and exits, *yes* then displays the read evaluate print (REPL) console.
--eval " *expression* "	evaluates an expression (in double quotes) and emits the result.
--grok *url*	reads and evaluates the the contents of the url.
--use *url*	attaches to a knowledge base for the session.
--help	displays the man page.
--logs *yes* | *no*	Toggles session logging. (*yes* is default) .

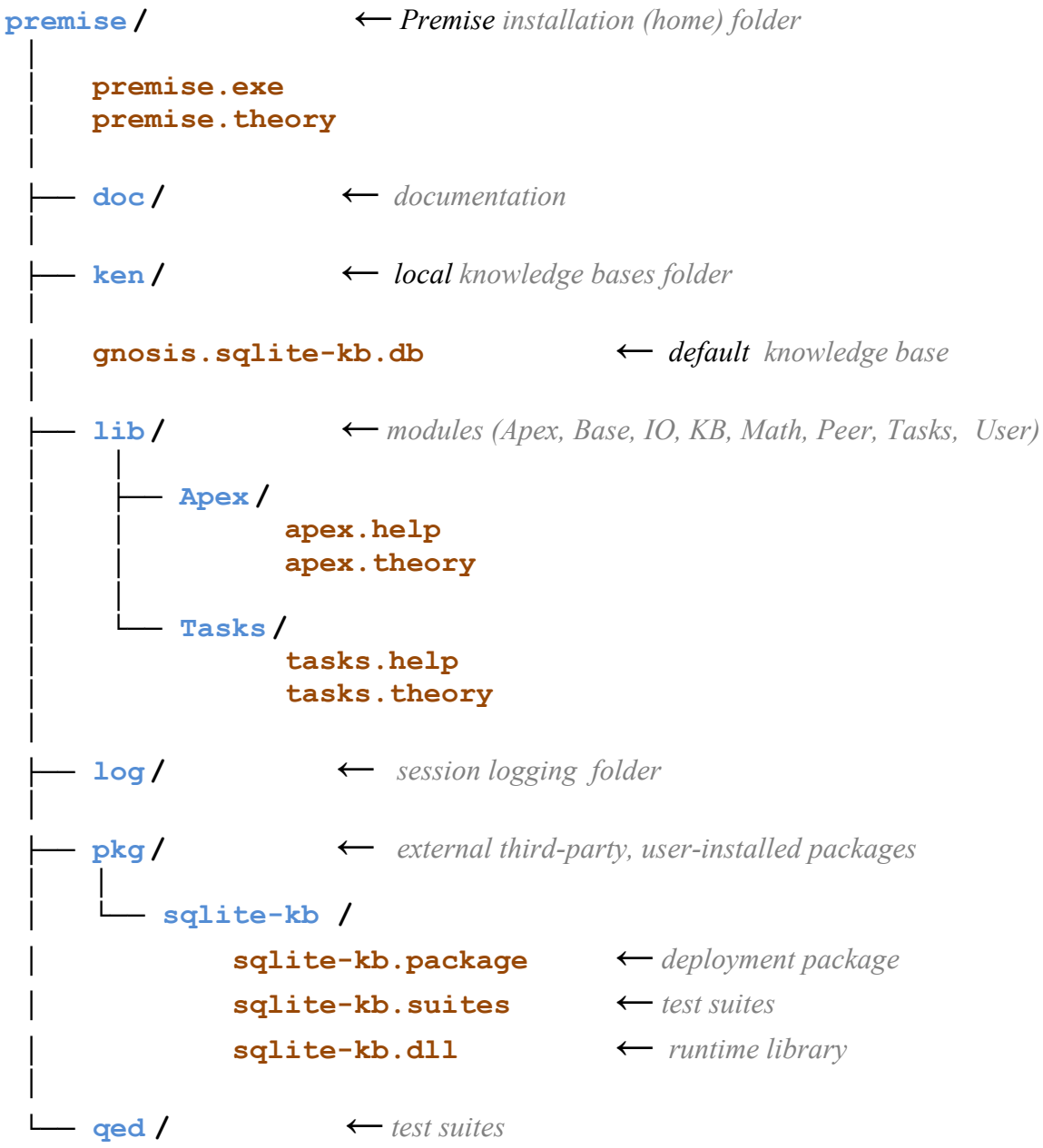

Figure 1. Premise folder hierarchy

To configure itself, Premise groks the ***premise.theory*** file located in the home folder. Foundational libraries are located in the **lib/** sub folder and third party packages are located in the **pkg/** sub-folder.

2. The Interpreter

The goal of Premise is to facilitate fast persistent relationship formation in a client server environment. Clients connect to the server and submit expressions which are interpreted and executed on the abstract machine. The expressions create and modify records in the knowledge store. Inference rules, if any are created, are executed by the inference engine.

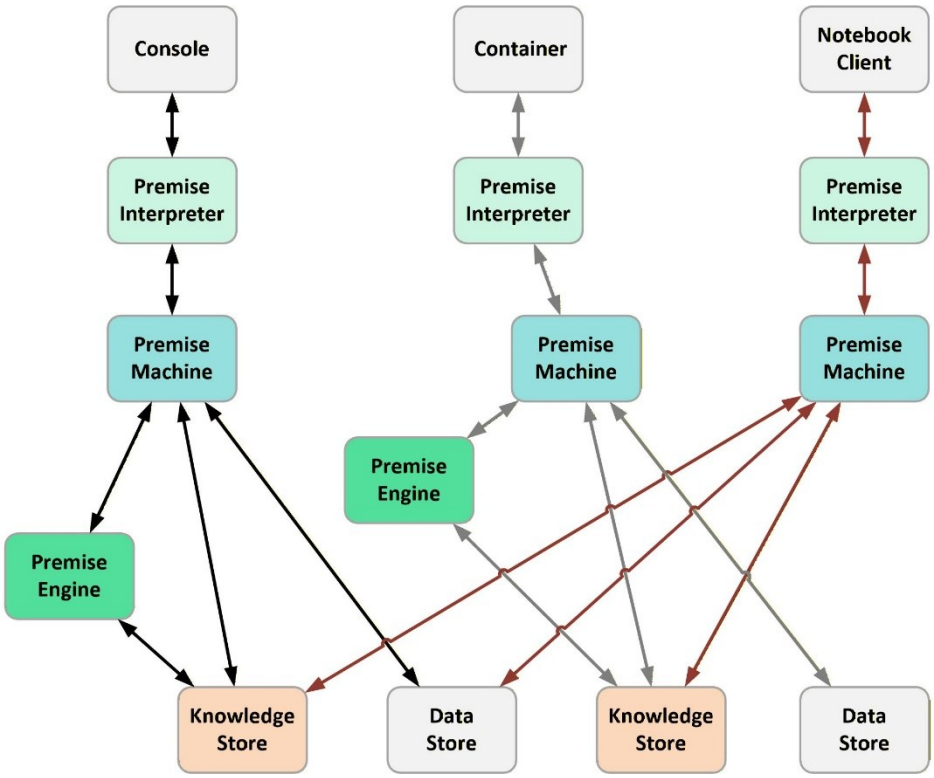

Figure 2. The Premise Interpreter Architecture

The Premise Language interpreter (**REPL**) follows several guiding principles, among them:

1. Metacircular evaluation. Parts of Premise are written in the Premise Language itself.

2. A Lisp-1 approach to symbol and function lookup is employed. This means that variables and functions can share the same symbol environment. Because variables are prefixed by question marks and functions are not, the difference between the symbol "?foo" and the function "foo" is evident. This eliminates one source of complexity for Lisp like languages.

3. The main point of The Premise Language is to build shared structures. To that end, relations, premises, and certain containers are persisted in an object store (e.g., a RAM database) to facilitate pattern matching over persisted records, and to facilitate concurrent access to shared structures.

The Read Evaluate Print Loop

The Premise Language interpreter, otherwise known as the Read Evaluate Print Loop, allows users to enter expressions at the incept (i.e., "> "). The interpreter evaluates the expression and returns the result. The result is printed after the ergo sign (i.e., " .: "). For example, when the literal yes is entered at the incept, it evaluates to itself and is returned by the interpreter as follows:

```
> yes

.: yes
```

An expression that adds two numbers together would return the sum like this:

```
> (+ 1 2)

.: 3
```

To print "Hello World" one would type the following.

```
> (tell user "Hello, World. \n")

Hello, World.
.: told
```

Imperative Programming

The Premise Language facilitates imperative, functional, and declarative programming. For example, adding the numbers from one to ten in an imperative style could involve defining a symbol **?total** to store the result, and looping an iteration symbol from 1 to 10.

```
> (function sum {list ?numbers}
    (let ?total 0)                ; assume ?total is 0
    (for ?n in ?numbers
      (set ?total (+ ?total ?n))) ; set ?total to the result of adding ?total and ?n
    ?total)                       ; return ?total

.: sum

>  (sum {1 2 3 4 5 6 7 8 9 10})   ; test the function

.: 55
```

Functional Programming

Adding numbers in a functional style might involve generating a list of numbers, walking through the list, and adding the numbers until we reach the end of the list. This approach eliminates the need for a **?total** symbol since the result is recursively kept on the program stack.

```
>  (function sum1 {list ?numbers}
     (if (void-p ?numbers)           ; if the list of numbers is empty
         0                           ; return zero
      else
         (+ (@ ?numbers)             ; add the first number
            (sum (rest ?numbers))))  ; to the sum of the remaining numbers
   )

.: sum1


>  (range 1 to 10)                  ; generate a list of numbers

.: {1 2 3 4 5 6 7 8 9 10}


>  (sum1 (range 1 to 10))           ; test the function

.: 55
```

An even more concise approach to sum uses the function **reduce**.

```
>  (function sum2 {list ?numbers}
    (reduce ?numbers +)
   )

.: sum2


>  (sum2 (range 1 to 10))

.: 55
```

Declarative Programming

Declarative programming involves establishing a set facts and writing a query or rule to utilize the facts. In The Premise Language, this approach utilizes relation instances, called "thoughts."

```
> (relation  Addend               ; make an Addend relation
    :Value
    :Counted  no
  )

.: Addend

> (step ?i from 1 to 10
    (new Addend :Value ?i))       ; make the individual addends

.: Addend_10

> (relation  Total                ; make a total relation
    :Result   0
  )

.: Total

> (new Total)                     ; make a new total

.: Total_1

> (with
    [Total  ^ as ?total]                                    ; match the Total
    [Addend ^ as ?addend :Value as ?value :Counted = no] ; match uncounted
   do
    (put ?addend :Counted yes)    ; indicate that the addend was used
    (==> ?total + :Result ?value) ; apply + to :Result and ?value, put in :Result slot
    (:Result ?total))             ; return the :Result slot value

.: 55
```

3. Taxonomy

Traditional programming languageve types and values. The values are often objects or scalars. The types are usually organized into a type **hierarchy** of abstract and concrete data types. The abstract data types, for example, Object, are placeholders to group subcategories of child data types into an inheritance tree. The concrete data types are what the programmer can actually create instances of.

In Premise, there are taxons and values. The data types are are called **taxons**. Taxons are organized into four important arrangements: a **taxonomy**, an **ontonomy**, a **meronomy**. and a **dexonomy**. The taxonomy defines the relationship among the data types starting at the most abstract taxon called a **variant** and descending to concrete taxons at the leaves of the arrangement (Figure 3). The immediate children of variant are **nothing** and **thing**. Nothing is concrete, while thing is abstract. Abstract taxons like variant, thing, time, and number, cannot be directly created in the Premise language while concrete taxons such as integer, string, list, and so on, can be created in the language

The Premise programmer is able to construct their own taxons in the language as well. These custom taxons are called **bespokes**, **enumerations**, and **prototypes**. Bespokes are keyword groupings of individual words, e.g., (bespoke Color red green blue). Enumerations are sets of words with corresponding ordinal numbers, e.g., (enumeration Priority high 10 medium 5 Low 1). Prototypes are resources that contain slots which are attributes, properties, or methods. There are two kinds of prototypes, **relations** which are persisted, and **structures** which are not persisted. For example, a car might be represented by the relation Car as follows: (relation Car :Make :Model :Year). Instances of prototypes are called **records**, and persisted records are called **thoughts** while non-persisted records are called **ephemerons.**

Prototypes exist in arrangements separate from the taxonomy. A subtype-supertype arrangement is called an **ontonmy**; a part-whole arrangement called a **meronomy**; and a membership-non membership arrangement is called a **dexonomy.** While the taxonomy is preconstructed, the ontonomy, meronomy, and dexonomy are dynamically constructed. The programmer is free to construct whatever other relationships they can conceive of using the available taxons of the Premise language.

The **ontonomy** is an arrangement of prototypes in *subtype-supertype* relationship to one another. The subtypes are called **kinds** and the supertypes are called **types.** For example an apple can have kinds such as Granny Smith, Fiji, or Golden Delicious. An apple can have types such as fruit, food, spherical object, and so on. A specific prototype can participate in multiple ontoonomic relationships simultaneously.

The **meronomy** is another arrangement of prototypes in a *part-whole* relationship to one another. There are **parts** and there are **wholes**. For example, an automobile has parts such as engine, wheels, and chassis. An automobile can have wholes such as a fleet, product line, race, traffic, and so on. A prototype can participate in multiple meronomic relationships simultaneously.

The **dexonomy** is another arrangement of prototypes in an *inclusion-exclusion* relationship to one another. Deixis is the way language points to people, places, or things in the immediate environment. A dexonomy is an arrangement of instances that are included in or excluded from categories, specifically, examples or counterexamples of a category.

```
variant — an abstract value of any type \%variant
├── nothing — a concrete value representing nothing \%nothing
│   ├── nil — a concrete unspecified value \%nil
│   └── void — a concrete value denoting emptiness \%void
└── thing — an abstract value representing something \%thing
    ├── atom — an abstract simple thing \%atom
    │   ├── number — an abstract number \%number
    │   │   ├── big — a concrete integer longer than 128 bits \%big
    │   │   ├── complex — a concrete number with real and imaginary parts \%complex
    │   │   ├── float — a concrete 128 bit floating point number \%float
    │   │   ├── imaginary — a concrete coefficient for the square root of -1 \%imaginary
    │   │   ├── integer — a concrete 64 bit signed integer \%integer
    │   │   ├── long — a concrete 128 bit signed integer \%long
    │   │   ├── numeric — a concrete numeric value, one of { \%infinity \%ninfinity \%indeterminate } \%numeric
    │   │   ├── rational — a concrete number as numerator over denominator \%rational
    │   │   ├── real — a concrete decimal number in scientific notation \%real
    │   │   ├── mezo — a concrete 32 bit unsigned integer number \%mezo
    │   │   ├── puny — a concrete 16 bit signed integer number \%puny
    │   │   ├── short — a concrete 32 bit signed integer number \%short
    │   │   ├── tiny — a concrete 16 bit unsigned integer number \%tiny
    │   │   ├── teeny — a concrete 8 bit unsigned integer number \%teeny
    │   │   ├── unsigned — a concrete 64 bit unsigned integer \%unsigned
    │   │   ├── uuid — a concrete 128 bit unsigned universal unique identifier \%uuid
    │   │   └── wee — a concrete 8 bit signed integer \%wee
    │   ├── time — an abstract measurement \%time
    │   │   ├── instant — an abstract single point in time \%instant
    │   │   │   ├── date — a concrete UTC date \%date
    │   │   │   ├── epoch — a concrete Unix epoch \%epoch
    │   │   │   ├── jiffy — a concrete millisecond 0.001 seconds \%jiffy
    │   │   │   ├── moment — a concrete nanoyear 0.0031573275 seconds \%moment
    │   │   │   ├── seconds — a concrete second \%seconds
    │   │   │   ├── tick — a concrete nanosecond \%tick
    │   │   │   └── temporal — a concrete time value, one of { \%eternity \%neternity \%indefinite } \%temporal
    │   │   └── interval — a concrete period spanning two time instants \%interval
    │   └── identifier — an abstract non-delimited case insensitive string \%identifier
    │       ├── symbol — a concrete question prefixed mutable variable \%symbol
    │       │   └── constant — a concrete question prefixed immutable constant \%constant
    │       ├── reference — a concrete pointer to a variant \%reference
    │       └── literal — a concrete undelimited alphanumeric sequence \%literal
    │           ├── unad — an abstract singleton \%unad
    │           │   ├── reserved — a concrete singleton member of a keywords group \%reserved
    │           │   └── coined — a concrete singleton member of a bespoke group \%coined
    │           ├── ordinal — a concrete name to number pair \%ordinal
    │           ├── slot — a concrete colon prefixed property identifier \%slot
    │           ├── function — a concrete literal that defines a function \%function
    │           │   ├── anonym — a concrete anonymous function \%anonym
    │           │   ├── esonym — a concrete internally named function \%esonym
    │           │   ├── exonym — a concrete user named function \%exonym
    │           │   ├── external — a concrete external function \%external
    │           │   ├── facility — a concrete private function \%facility
    │           │   ├── generator — a concrete function that creates a series \%generator
    │           │   ├── intrinsic — an concrete language primitive \%intrinsic
    │           │   ├── junction — a concrete parallel evaluation function \%junction
    │           │   ├── macro — a concrete user named macro \%macro
    │           │   ├── method — a concrete exclamation prefixed function literal \%method
    │           │   ├── predicate — a concrete Boolean function returning a Boolean value \%predicate
    │           │   ├── procedure — a concrete function returning nothing \%procedure
    │           │   ├── rule — a concrete rule \%rule
    │           │   └── thunk — a concrete evaluation delay \%thunk
    │           └── resource — an abstract resource location \%resource
    │               ├── url — a concrete universal resource \%url
    │               │   ├── data — a concrete database connection \%data
    │               │   ├── file — a concrete file resource \%file
    │               │   ├── folder — a concrete folder resource \%folder
    │               │   ├── index — a concrete relation access resource \%index
    │               │   ├── knowledge — a concrete knowledge base connection \%knowledge
    │               │   ├── port — a concrete port \%port
    │               │   └── pipe — a concrete input output resource \%pipe
    │               ├── compiler — a concrete expression compiler \%compiler
    │               ├── engine — a concrete inference engine \%engine
    │               ├── evaluator — a concrete expression evaluator \%evaluator
    │               ├── grimoire — a concrete grammar for grokking and emitting \%grimoire
    │               ├── interpreter — a concrete expression interpreter \%interpreter
    │               ├── machine — a concrete Premise machine \%machine
    │               ├── reader — a concrete expression reader \%reader
    │               ├── writer — a concrete expression writer \%writer
    │               ├── prototype — an abstract relationship \%prototype
    │               │   ├── relation — a concrete persisted relationship \%relation
    │               │   │   └── problem — a concrete thought grouping \%problem
    │               │   └── structure — a concrete ephemeral relationship \%structure
    │               ├── record — an abstract record \%record
    │               │   ├── thought — a concrete persisted record \%thought
    │               │   └── ephemeron — a concrete ephemeral record \%ephemeron
    │               │
    │               └── task — a concrete task resource \%task
    │                   ├── service — a concrete service \%service
    │                   └── agent — a concrete agent \%agent
    │
    │
    └── container — an abstract container \%container
```

Figure 3. Premise Taxons

Taxons

In Premise, all values are typed. These types are called taxons. Variables (also called symbols) are bound to typed values, but the symbols themselves are not typed.

Noumenons

A noumenon is the raw textual representation of a value. Typically this representation will have this format: backslash, type, subtype, payload. There are four noumenons, taxon (**%**), number (**#**), time (**@**), reference (**&**). Subtypes can be one or more characters (e.g., m - moment, id - integer decimal, literal – the literal taxon). The payload is delimited by curly braces, **{ }**. Some examples are as follows.

```
\%yes    \%no     \#id{+9821}      \#id{0}     \@m{12345}     \@e{47}
\@d{1990-01-10T09:30:00.001Z}     \%literal{foo}    \&{?x}    \&{12345}
```

Variants

A variant is a value that can be any type. Variants are either nothing or something (i.e. a thing).

Nothing

The taxon **\%nothing** which evaluates to the literal **nothing** represents values that do not exist. The taxon **\%nil** represents something that is unspecified. It can be compared to the literal **nil** . Unspecified values default to nil with the **is** function. A value of **\%void** means a container has no elements. The function **null-p** can be used to determine whether a value is either unspecified or nonexistent. Containers default to empty and can be tested with the **void-p** function.

```
\%nothing        \%nil            \%void
```

Things

The type **thing** represents a value that exists. The function **thing-p** can be used to detect if a value is a thing. The thing type is subdivided into **atom** and **container**.

Atoms

An atom is a number, time value , or identifier (symbol or literal).

Numbers

The number data type encapsulates several sub types: big, integer, long, unsigned, rational, real, float, imaginary, and complex.

Integers

Positive and negative decimal integers up to 64 bits are supported. Apostrophes (') used as separators within numbers are ignored.

```
-5                                  \#id{+3}
+999999999999999999                 \#id{-999'999'999'999'999'999}
```

Radix

Integers from radix 2 through radix 36 of the form \#TDD**r{**SN…} are supported, where T is a subtype indicator (n:Integer, g:Long, b:Big, r:Real, f:Float, i:Imaginary, u:Unsigned, c:Complex), D is a decimal number between zero and nine, s is an optional sign for positive (+) or negative (-) number, and N is a number from zero (0) to nine (9) or an uppercase letter from A through Z. The radix combination DD must be within the range of 2 to 36 inclusive and the digits N must be less than DD. For example #2r has digits 0 and 1 while #16r has digts 0 through F.

```
\#i02r{+10110}          \#i16r{-778CF20}          \#i36r{+84QQN250MLQZ}
```

Binary

Binary integers (\#i2r) are supported in the abbreviated form #\nbSN… for N equals 0 or 1.

```
}          \#ib{+1010}              \#ib{-10101}              \#ib{0}
```

Octal

Octal integers (\#i8r) are supported in the form #noSN… for N from 0 to 7.

```
           \#io{+1234}              \#io{+0}                  \#io{+72'342}
```

Decimal

Decimal integers (\#i10r) are supported by default and also in the form \#idSN… for N from 0 to 9.

```
           \#id{+9821}              \#id{+0}                  \#id{-8'290'553}
```

Hexadecimal

Hexadecimal integers (#n16r) are supported in the form #nxSN… for N from 0 to F.

```
           \#ix{-00FF}              \#ix{0}                   \#ix{82'9CA'FF3}
```

Hexatridecimal

Hexatridecimal integers (\#i36r) are supported in the form \e#nzSN… for N from 0 to Z.

```
           \#iz{-QA5N}              \#iz{0}                   \#iz{+7'0M6'Z8P}
```

Big

A big number is a decimal integer that is longer than 128 bits.

```
        \#bd{+100000000000000000000}      \#bd{0}  \#bd{-80000000000000000000000000000000}

        \#bd{23928374928379658218798105687251871972938729387492837492876582736482763876283762487364827364872638476827368427634872638476287482364827634872638476287634762876348273648726384768247682376482763482736482763847268347628376487684}
```

UUID

Universally unique identifiers (UUIDv7 RFC9562 standard) of 128 bits prefixed by the letters "uu".

```
   \#uu{+018f4a8d7c9179d3b6a4c2e8f1d7b5a9}   \#uu{+018f4a8d7c9174f8a2c1e9b6d3f7a5c}
   \#uu{+018f4a8d7c917b529a7d4e3f6c8b1d25}   \#uu{+018f4a8d7c9173a19b4c6d2e8f1a3c7}
```

Longs

A long number is a decimal integer that is 128 bits.

```
        \#nd{+100000000000000000000}          \#nd{-800'000'000'000'000'000'000}
```

Unsigned

Unsigned 64 bit integers y are prefixed by the letter "u"

```
        \#ud{+0}                                  \#ud{+3}
        \#ud{+18446744073709551615}               \#ud{+99'999'999'999'999'999}
```

Integer

Signed 64 bit integers are prefixed by the letter "i"

```
123                                 -999
\#id{+0}                            \#ib{+111010010010011}
\#iz{+184QG3U2709JL55615}            \#ud{+99'999'999'999'999'999}
```

Mezo

Unsigned 32 bit numbers are prefixed with the letter "m".

```
\#mb{10101110101}          \#md{+780}          \#md{4292967296}
```

Short

Signed 32 bit numbers are prefixed with the letter "s".

```
\#sb{+11011011101111}      \#sd{0}                  \#sd{32765}
```

Tiny

Unsigned 16 bit integers are prefixed with the letter "t".

```
\#td{+10}                  \#td{128}                \#tb{11111111}
```

Puny

Signed 16 bit integers are prefixed with the letter "p".

```
\#pd{+10}                \#pd{128}            \#pb{11111111}
```

Teeny

Unsigned 8 bit numbers are prefixed with the letter "z".

```
\#zd{10}                 \#zd{126}                \#zb{1111111}
```

Wee

Signed 8 bit numbers are prefixed with the letter "w".

```
\#wb{+01110101}          \#wd{+80}                \#wd{-25}
```

Rational

Rational numbers (quotients) have 64 bit integer numerator and denominator prefixed with "q"..

```
3/4                                          \#qd{+3 +4}
-8/20                                        \#qd{-8 +20}
```

Float

Float numbers are 128 bits, have high precision and a smal range. Floats can be used for monetary values. Float calculations must be made explicit by wrapping them in the float function , e.g. (+ (float 100) (float 200)), or prefixing \#fd before the number.

```
\#fd{-98'768'762'983.237876867749}        (float -2342.0e-23)
\#fd{+234.0e+0}                           \#fd{124.95}
```

Real

Real numbers are 64 bit numbers in scientific notation, prefixed with "r".

```
10.0                                      (real 0.001)
\#rd{+2342.243}                           \#rd{-54.50}
```

Imaginary

Imaginary numbers are 64 bit floating point number suffixed by the letter "y" (representing the square root of **-1**).

```
\#yd{+21}          \#yd{-5.0}       \#yd{2.9}        \#yd{-9.93}
```

Complex

Complex numbers include a 64 bit floating point real part plus a 64 bit floating point imaginary part.

```
\#cd{+55 +34}           \#cd{-2.35 +7}          \#cd{-32 -14.3}
\#cd{0 -2}              \#cd{75 +1}             \#cd{0 +1}
```

Numerics

A numeric represents any of the values for infinity, negative infinity or indeterminate numbers.

```
\%numeric
```

Infinity

The infinity taxon represents a positive infinite number.

```
\%infinity
```

Ninfinity

The ninfinity taxon represents a negative infinte number.

```
\%ninfinity
```

Indeterminate

The indeterminate taxon represents an impossible or undefined number.

```
\%indeterminate
```

Time

Time is supported in several varieties: dates, epochs, intervals, moments, seconds, and ticks.

Dates

A date represents time as a universal time coordinate (YYYY-MM-DDThh:mm:ss.sZ) in Zulu time or with an offset.

```
\@d{1990-01-10T09:30:00.001Z}     \@d{2016-11-24T08:15:30-08:00}
```

Epochs

The epoch data abstraction represents time in unix epochs as a 128 bit integer value prefixed by the letter "e". An epoch represents the number of seconds since midnight January 1, 1970.

```
\@e{10}                  \@e{592}                 \@e{60}
```

Jiffies

The jiffy data abstraction represents time as milliseconds.

```
\@j{-12}                 \@j{+600}                \@j{60}
```

Moments

The moment data abstraction represents time in nano-years (approximately 0.0031573275 seconds) as a 16 digit, 64 bit, integer value prefixed by the letter "m".

```
\@m{2025500000000000}            \@m{-1}
\@m{2012001005050000}            \@m{+3000}       ; approximately 10 seconds
```

Seconds

The seconds data abstraction represents time in seconds as a 64 bit integer value prefixed by the letter "s".

```
\@s{10}                    \@s{592}                    \@s{60}
```

Ticks

The tick data abstraction represents time in nanoseconds as a 128 bit integer value prefixed by the letter "t".

```
                          \@t{2342}                \@t{100000}
```

Intervals

The interval data abstraction represents a time coordinate as a pair of time instants. The lesser time instant is followed by the greater time instant.

```
\@i{\@m{2012001005050000} \@m{2025500000000000}}

\@i{\@s{3} \@s{10}}       \@i{\@d{2012-05-19} \@d{2013-07-23}}
```

Temporal

The temporal taxon represents eternity, negative eternity, impossible, or indefinite time.

```
\%temporal
```

Eternity

The eternity taxon represents an infinite amount of time in the future.

```
\%eternity
```

Neternity

The neternity taxon represents an infinite amount of time in the past.

```
\%neternity
```

Indefinite

The indefinite taxon represents an undefined or impossible time. Usually the default for an instant.

```
\%indefinite
```

Identifiers

An identifier is an alphanumeric sequence of characters not delimited by quotes.

Symbols

A symbol is a variable, i.e., an identifier that has both a name and an associated value. In the Premise Language, symbols are literals starting with a question mark and ending with a sequence of numeric or alphanumeric characters. When symbols are declared their value is immediately set to an initial value. Referencing an undeclared symbol will cause a failure. Symbols are typically written in lowercase. Dots in the symbol name indicate a symbol within a particular module. For example:

```
?height          ?User.width            ?length              ?2
```

Constants

A constant is a symbol whose value cannot change. By convention constants are written in uppercase.

```
?MAX-LIMIT       ?WIDTH-MIN             ?*DEFAULT-LENGTH*
```

References

A reference is a pointer to a variant. A reference allows a value to be passed as a reference parameter. A reference is printed as \& and the symbol name (e.g., **\&{?**me}) or value otherwise (e.g., **\&{"fox"}**, **\&{42}**).

```
(reference ?MAX-LIMIT)           (reference ?2)

\&{?MAX-LIMIT}                   \&{?2}
```

Symbol Scoping

Symbol scope refers to the environment a symbol belongs to. A symbol may have one of five scopes: global, module, lexical, dynamic, task, and restricted.

Global Scope

Global variables are declared using the **global** function.

```
> (do (global ?X 1)
      (function g (tell user ($$ ?X \s)) (set ?X 2))
      (function f (global ?X 3)(g)))
      (f)
      (tell user ($ ?X ($))))

3 2 .: told
```

Module Scope

A symbol may be scoped to the current module using the **modular** function.

```
> (do (global ?X 1)
      (function g (tell user ($$ ?X \s)) (set ?X 2))
      (function f (modular ?X 3) (g)))
      (f)
      (tell user ($ ?X ($))))

3 1 .: told
```

Lexical Scope

Lexical scoping in Premise is achieved via the **so**, **let**, and **var** intrinsics. Multiple tasks may access the same lexical symbol. The **so** intrinsic creates a new scope, while **let** and **var** add variables to the existing scope.

```
> (do (global ?X 1)
      (function g (tell user ($$ ?x \s)) (set ?x 2))
      (function f (so {?x 3} (g)))
      (f)
      (tell user ($ ?X ($))))

3 1 .: told

> (do (global ?X 1)
      (function g (tell user ($$ ?x \s)) (set ?x 2))
      (function f (let ?x 3) (g)))
      (f)
      (tell user ($ ?X ($))))

3 1 .: told

> (do (global ?X 1)
      (function g (tell user ($$ ?x \s)) (set ?x 2))
      (function f (var ?x 3) (g)))
      (f)
      (tell user ($ ?X ($))))

3 1 .: told
```

Dynamic Scope

A **dynamic** scope creates temporary global variables.

```
> (do (global ?X 1)
      (function g (tell user ($$ ?X \s)) (set ?x 2))
      (function f (dynamic {?x 3} (g)))
      (f)
      (tell user ($ ?X ($))))

3 2 .: told
```

Task Scope

Variables **local** to a particular task are shielded from changes by other tasks.

```
> (do (global ?X 1)
      (function g (tell user ($$ ?x \s)) (set ?x 2))
      (function f (local {?x 3} (g)))
      (f)
      (tell user ($ ?X ($))))

3 1 .: told
```

Restricted Scope

Restricted scopes **only** shadow variables and prohibit free variables.

```
> (do (global ?X 1)
      (function g (tell user ($$ ?x \s)) (set ?x 2))
      (function f (only {?x} (g)))
      (f)
      (tell user ($ ?X ($))))

1 1 .: told
```

Symbol Assignment

Variables may be bound to values in a variety of ways.

Fix

The **fix** intrinsic declares taxon restricted variables in the current environment, returning yes.

```
> (fix integer ?age 10 string ?name "John Doe" literal ?fruit  grape)
.: true
> ?age  ?name  ?fruit
.: 10 "John Doe" grape
```

Let

The **let** intrinsic declares and initializes variables in the current environment, returning yes.

```
> (let ?X 1 ?Y 2 ?Z 3)
.: true
> (let {?a ?b ?c} {dog cat fish})
.: true
> ?a ?b ?c ?X ?Y ?Z
.: dog cat fish 1 2 3
```

Var

The **var** intrinsic declares and initializes variables, returning the value of the last assignment.

```
> (var {?a ?b ?c} {dog cat fish} ?X 1 ?Y 2 ?Z 3)
.: 3
> ?a ?b ?c ?X ?Y ?Z
.: dog cat fish 1 2 3
```

Bind

The **bind** intrinsic declares variables in the current environment and binds them to object property values, object method values, sequence positions, or functions, and then returns **yes**.

```
> (relation Counter
    :Value 0  :Resets 0 :Reclaim  no
  )

.: Counter

> (new Counter)

.: Counter_1

> (bind Counter_1 :Value ?value)

.: true

> ?value

.: 0

> (bind {a b c d e f g}  1 ?first # ?last)

.: true

> ?first ?last

.: a g
```

Set

The **set** intrinsic updates declared variables, and returns **yes**.

```
> (set ?X 1 ?Y (++ ?X) ?Z (++ Y))

.: true

> ?X  ?Y  ?Z

.: 1 2 3
```

Tie

The **tie** intrinsic updates declared variables, returning the last assigned value.

```
> (tie ?X 1 ?Y (++ ?X) ?Z (++ ?Y))
.: 3
> (tie {?X ?Z} {2 2})
.: 2
> ?X  ?Y  ?Z
.: 2 2 2
```

<-- Before Tie

The **before tie** intrinsic captures a symbol's return value before binding the symbol to an expression.

```
> (global ?X 1 ?Y 1 ?Z 1)
.: true
> (<-- ++ ?X)
.: 1
> ?X
.: 2
```

--> After Tie

The **after tie** intrinsic captures a symbol's return value after binding the symbol to an expression .

```
> (global ?X 1)
.: true
> (--> + ?X 1)
.: 2
> ?X
.: 2
```

Literals

Literals are non-numeric constants which evaluate to themselves. Typically, literals begin with an alphabetic or punctuation character (such as colon or exclamation point), and contain contiguous sequences of alphabetic, numeric, underscore or period characters. Literals provide names for functions, relationship prototypes, enumeration values, and so forth. Some examples of literals are as follows:

```
a               one             :2              three
a_1             a_12-34         hello-world     the_quick_brown_fox
*               !               :=              -a-curious-literal
```

Structures

Structures are unpersisted ephemeral relationships, created with the **structure** function.

```
> (structure Fault   :What  :Where  :When)

.: Fault

> (structure Message  :Sender :Recipient :Content)

.: Message
```

Slots

Slots are literals preceded by a colon. Some examples are:

```
:Height          :Width          :Length          :321              ^
```

The identifier slot is the caret character: **^** .

Slots hold data values. In relation definitions, slots are followed by values called **defaults**. In instances, slots are followed by actual values called **terms**. When no default or referent is present for a slot, the value **nil** is implied.

The following are examples of relationship prototypes with slots and defaults.

```
(structure Red
  :Object corvette
  :Size little )


(structure Phrase
  :Saying "Who goes there"
  :Count  14 )
```

The following are some examples of premises with slots and terms.

```
[Red ^ Red_56 :Object corvette :Size litte]

[Event ^ Event_101 :Subject bird]

[Basket ^ Basket_12 :Items {apple banana orange}]

[Person ^ {Person_201 Person_202 Person_203} :Name "John Smith"]
```

When a call has a relation slot as its first item and a premise or thought as the second item, the call will return value of the slot within the premise or from the thought.

```
> (:Color [Fruit :Color Green])

.: Green
```

```
> (:Quantifier Fact_35)

.: Some
```

Relations

A persisted relationship is comprised of a type literal, slots, methods, and default values. A thought premise has a relation literal, slots, methods, and actual values. Relationships are defined using the **relation** function. Thoughts are created using the **new** or the **knew** function. Finally, there are three kinds of slots for a Relationship: data slots (prefixed with a colon), method slots (prefixed with an exclamation point), and identity slots (using the caret character to name the slot).

The **relation** function is used to define a relationship. The relation definition starts with a left parenthesis, followed by the literal **relation**, followed by an identifier for the type name, followed by any relation dependencies (each dependency is preceded by the **uses** keyword), then the slots and default values, methods and function names, and finally ending with a right parenthesis.

A relationship prototype must be defined before a new thought can be formed. Consider the following definitions:

```
> (relation Fruit
    :Color
    :Weight  1.0
    !eat     Fruit_eat
  )

.: Fruit
```

```
> (relation Apple
    uses Fruit
    :Color  red
  )

.: Apple
```

```
> (relation Delicious
    uses Apple
    :Color yellow
  )

.: Delicious
```

The Identity Slot

Thoughts have an identity slot (^) that contains the identifier value which uniquely demarcates the thought. Some examples of thought identifiers are

```
Sentence_1                  Red_34                  SOV_74

Vocalization_9202           Instance_983
```

Thought premises also contain an identity slot. For example,

```
> (premise Red_1)

.: [Red  ^ Red_1  :Object  corvette :Size little]

> (premise Event_29)

.: [Event  ^ Event_29 :subject bird]
```

A self reference value can be used when forming thoughts using the **new** function. Self (**\^**) is equivalent to a "this" symbol in some structured languages such as C++, or the ?me symbol in Premise.

```
(new Car :make Dodge :model RAM :year 2010 :id \^)
```

Methods

Methods are literals preceded by an exclamation point. Some examples are:

```
!action          !cleanup         !onNew           !onOld
```

Relationships have several built in methods which can be implemented by the developer: a constructor (!onNew), destructor (!onOld), copy method (!onCopy), and an update method (!onSet). The **!onNew** method initializes a premise. The **!onOld** method disposes of a premise.

Calling a method will look up the function name located in the method and invoke the function on the arguments. The methods cloneMe and Concept_Add will be retrieved when invoked.

```
[MyObject  !save   saveMe   !clone  cloneMe]
[Concept   :Datum  0  !add  Concept_add ]
```

There are two ways of invoking a method. Methods are invoked using the call function or by using the method name as a function (i.e, by placing the method name first in a call followed by the instance containing the method). A method not found failure will be thrown if the instance does not contain the method.

```
(call  !start  car_255  ?myKey)        ; invoking a method using the call function.
(!start  car_255   ?myKey)             ; invoking a method using the method name.
```

When a call has a method as its first item and a premise or identifier as the second item, the call will return the value of applying the method to the remaining arguments.

```
> (relation Foo
    :Datum 100
    !add2  Foo_addTwo
  )

.: Foo

> (function Foo_addTwo {?me}
    (==> ?me + :Datum 2)
  )

.: Foo_addTwo
```

```
> (new Foo :Datum 1000)

.: Foo_1

> (:Datum Foo_1)

.: 1000

> (!add2 Foo_1)

.: 1002
```

Resources

A resource is a literal which represents a tuple. Resources can be created for tasks, files, assortments, data connections, and so forth. Typically a resource will indicate the type of resource, followed by a hyphen, followed by an integer resource number.

```
File-45                 Task-32                 Data-988
Lexicon-87              Environment-67          Colors
```

Assortments

Assortments are resources that hold unordered elements, associations, bindings, or enumerations. Assortments such as collections, lexicons, environments, and enumerations are provided along with specific functions to manipulate them (e.g., add, cut, item, keys, put, the, tie, values, etc.).

```
> (lexicon q nil)

.: Lexicon-1

> (add Lexicon-1 "z" foo)

.: Lexicon-1

> (add Lexicon-1  84 nil)

.: Lexicon-1

> (entries  Lexicon-1)

.: {{q nil}{'z' foo}{84 nil}}
```

```
> (cut Lexicon-1 "z")

.: Lexicon-1

> (entries  Lexicon-1)

.: {{84 nil}{q nil}}

> (lexicon :Length 24 :Width 12 :Height 9 :Units inch)

.: Lexicon-2

> (@ Lexicon-2 :Length)

.: 24

> (@ Lexicon-2 :Width)

.: 12

> ?width

.: [Failure :Name UndefinedSymbol :Text "Symbol ?width is undefined."]

> ?height

.: [Failure :Name UndefinedSymbol :Text "Symbol ?height is undefined."]

> (bind Lexicon-2 :Width ?width :Height ?height)

.: true

> ?width

.: 12


> ?height

.: 9

> (keys Lexicon-2)

.: {:Height :Length :Units :Width}

> (values Lexicon-2)

.: {inch 12 24 9}

> (enumeration Colors
    red orange yellow green blue indigo violet)

.: Colors

> (enum Colors red)

.: 1

> (keys Colors)

.: {red orange yellow green blue indigo violet}

> (values Colors)
```

```
.: {1 2 3 4 5 6 7}

> (enum Colors green)

.: 4

> (entries Colors)

.: {{red 1}{orange 2}{yellow 3}{green 4}{blue 5}{indigo 6}{violet 7}}

> (enumeration Controls

    clock 100
    timer 200
    start 300)

.: Controls

> (enum Controls timer)

.: 200

> (keys Controls)

.: {clock start timer}

> (collect ?key ?value per (sort (entries  Controls) {< 2}) ?key)

.: {clock timer start}
```

```
> (enumeration Highways

    Maine-Miami              ; defaults to 1
    Maine-Idaho              ; defaults to 2
    Michigan-Washington 10
    NewYork-Louisiana        ;will be 11
    Massachusetts-Oregon 20
    Ohfile-Florida             ; will be 21
    NewJersey-Oregon 30
    Michigan-Alabama
    NewJersey-California 40
    Michigan-Florida
    Maryland-Nevada  50
    Wisconsin-Louisiana
    Illinois-California 60
    Minnesota-Louisiana
    NorthCarolina-Arizona  70
    JeffersonHighway
    Georgia-California 80
    NorthDakota-Texas
    Florida-Texas 90
    Montana-Nevada
    Washington-California 101)

.: Highways

> (sort (values Highways) <)

.: {1 2 10 11 20 21 30 31 40 41 50 51 60 61 70 71 80 81 90 91 101}

> (enum Highways NewJersy-California)

.: 40
```

Collections

Collections are sets that contain only keys. They are created with the **collection** function.

```
> (collection
    the quick brown fox 21  "over the" lazy dogs [MyObject ^ MyObject_32] )

.: Collection-1
```

Configurations

Configurations are key value pairs that are persisted to a file. They are created with the **configuration** function.

```
> (configuration "file://localhost.com/c$/apps/Premise/"
    "/mind/agent/delay" \@m{35}
    "/mind/activation/history/maximum"  15)

.: Configuration-1
```

Environments

Environments contain identifier to value bindings. They are created with the **environment** function.

```
> (environment nil
    (symbol x) 32   times10 (given {?x} (* ?x 10)) )

.: Environment-1
```

Bespokes

Bespoke taxons are created by the User to extend the available taxons in the Premise milieu. Bespoke taxons contain **coined** taxons as elements. Each of which hhas a corresponding noumenon and can be used as taxon restrictions on functions.

```
> (bespoke favoriteColors
    chartreuse teale burgundy orange )

.: favoriteColors
```

Enumerations

Enumerations contain literal elements. Each element of the enumeration has a position. By default the first element will have position one, the second two, and so forth. If any of the arguments to the enumeration function is numeric, then that argument will set the position for the preceding literal. Enumerations are created with the **enumeration** function.

```
> (enumeration  Colors  red  orange   yellow )

.: Enumeration-1

> (enumeration Controls  clock 100 timer 200 start 300)

.: Enumeration-2
```

Ephemerons

Ephemerons are manifested structures created with the **new** function. A Fault ephemeron can be defined as follows. Because they are short lived, ephemerons are represented as premises, and do not have a unique identifier. They keys for an ephemeron are fixed and are defined by the structure the ephemeron instantiates.

```
> (new Fault :What  AnError :Where RightHere :When Now)

.: [Fault :What AnError :Where RightHere :When Now]

> (new Message :Sender Me :Recipient Todd :Content "Hello There")

.: [Message :Sender Me :Recipient Todd :Content "Hello There"]
```

Thoughts

A thought is record and an instance of a relation. Thoughts have predefined slots, and are added to memory using the **new** or **knew** function. A relation is a prototype that defines a fixed record structure. Each thought instance has a literal identifier. In the case below, Apple_1 and Delicious_34 are identifiers for thoughts.

```
> (new Apple :Weight 2.0)

.:  Apple_1

> (new Delicious)

.: Delicious_34
```

Lexicons

Lexicons contain value to value associations. They are created with the **lexicon** function.

```
> (lexicon
    {a b c} "123"  the     {"  t" " th" "the" "he " "e  "} :Every  body )

.: Lexicon-1

> (entries  Lexicon-1)

.: {{a b c} "123"}{the {"  t" " th" "the" "he " "e  "}}{:Every body}}
```

Continuations

Continuations enable jumps into functions. They are created with the **continuation** function.

```
> (function factorial_hps {?n ?k}
    (let ?a 1)
    (continuation ?m ?a)
    (if (<= ?n 0)
        (continue ?k ?a)
     else
        (--> -- ?n)
        (continue ?m (* (++ ?n) ?a))))

.: factorial_hps


> (function factorial {?n}
    (let ?r nil)
    (continuation ?k ?r)
    (if (nil-p ?r) (factorial_hps ?n ?k) else ?r))

.: factorial

> (factorial 5)

.: 120
```

Data

Data resources enable the use of information sources. They are created with the **data** function.

```
> (global ?Data (data driver sqlserver server "//servername" port 1433
                      database mydata user "myUid"  password "123456")

.: Data-1

> (ask ?Data "select top 2 empid, name, salary from employee")

.: {{1 "John Smith" 10.50}{2 "Jane Johnson" 12.00}}
```

Files

File resources enable the reading and writing of files. They are created with the **file** function.

```
> (write (file name "quick.txt" seek 1) "The quick brown fox")
.: File-1
> (close file-1)
.: closed
```

Tasks

Tasks are expressions that can be evaluated in separate threads of execution. They are created with the **task** function.

```
> (task (/ 1000 57) (** 4 20) (** 4 20))
.: {Task-1 Task-2 Task-3}
> (complete task-1 task-2)
.: {Task-1 Task-2}
> (await task-1)
.: 17.54385964912281
> (await task-2)
.: 1099511627776
> (concurrent task-3)
.: {Task-3}
> (await task-3)
.: 1099511627776
> (free {task-1 task-2 task-3})
.: freed
```

Indices

Index resources are used to increased the speed of instance retrieval. They are created with the **index** function.

```
> (relation Car :Make :Model :Year)

.: Car

> (index Car :Make :Model)

.: Car_Index_1

> (index Car :Model)

.: Car_Index_2

> (new Car :Make A :Model M1)

.: Car_1

> (with Car ^ as ?i :Model M1) list ?i)

.: {Car_1}

> (drop Car_Index_1)

.: dropped

> (drop Car_Index_2)

.: dropped
```

Reserved Literals

Certain literals are keywords that have a distinctive meaning. Keywords can optionally be prefixed with **\%** to be explicit about the use of the keyword. For example, the **Truth** reserved keywords represent the possible logical states.

```
true      false      paradox      neither      absurd      unknown

\%True    \%False    \%Paradox    \%Neither    \%Absurd    \%Unknown
```

Intrinsic Functions

The function literals that comprise the Premise Language are reserved and should not be redefined in their native modules since attempting to do so would redefine the behavior of the language.

Modules

Modules are name spaces. A module literal is used to avoid literal collisions and provide modularity in Premise code. Modules are delimited by dots (.) within a literal. There are several initial modules.

Apex	Global environment
Base	Foundational functions
IO	File & Messaging functions
KB	Knowledge access
Maths	Mathematic functions
Tasks	Task functions
User	Default environment

All modules are global, meaning that the modules cannot be concatenated or nested. Modules are declared with the **module** function, and modified with the **extend** function, and deleted with the **discard** function.

Containers

Containers are groupings of values. There are two kinds of containers, assortments and sequences. Assortments are unordered resource groupings. Sequences are ordered groupings of values.

Sequences

Sequences are ordered groupings of values. The main function of a sequence is to group values into arrays, sets, or bags of items. The Premise Language provides functions for accessing sequences as a series of elements. A sequence is a string, a list, a call, or a tuple. The language provides functions to manipulate sequences (e.g, # element, make, bind, for).

```
> (@ "ABC" 2)    ; find the second element of the string

.: "B"

> (@ {The quick brown fox jumped over the lazy dogs} 5)

.: jumped

> (copy {A B C} 1 X 3 Z)   ; copy list, replacing positions 1 and 3

.: {X B Z}

> (copy "ABC" 1 X 3 Z) ; copy string, replacing positions 1 and 3

.: "XBZ"

> ?v1

.: [Failure :Name UndefinedSymbol :Text "Symbol ?v is undefined."]

> (let {A B C} 1 ?v1 2 ?v2)

.: true

> ?v1

.: A

> (let "ABC" 1 ?s1 2 ?s2)

.: true

> ?s1 ?s2

.: "A" "B"

> (for ?c in "the quick brown fox"  (tell user ($ ?c)))

the quick brown fox .: told

> (for ?n in {1 2 3} (tell user ($$ (* ?n 10) \s )))

10 20 30 .: told
```

Expressions

Expressions are sequences of zero or more variants without delimiters. Expressions are usually found inside sequences such as lists, tuples, or calls.

```
an expression            don't do that
another expression       He said,  Sure why not?
```

Lists

Lists are ordered groupings of values which evaluate to themselves. Lists are malleable so they can be extended or reduced and elements can be inserted or deleted. This malleability allows lists to represent stacks or queues or orderings. The empty list, **{ }**, may be used in code for clarity. Lists are created with the **list** function.

```
{a b c}                  {1 2 3}                  {}
```

Vectors

Vectors are sequences of numbers delimited by pipes (vertical bar). Numbers are not supported as a distinct data type; instead a byte is an integer written in base 16.

```
|0|             |1 2 3|      |\#ix{FF} \#ix{E2} \#ix{C8}|  |\#nx{FFE2C872}|
```

Strings

Strings are sequences of characters delimited by double quotes. Characters are not supported as a distinct data type, instead characters are strings of length 1.

```
"a string"               "don't do that"
"another string"         "He said, 'Sure why not?'."
```

Backslashes provide character escaping. Doubling the backslash character within a string provides an escape for the backslash character. For Example \\ will represent a single \. Other escape characters are as follows:

Escape Sequences	Character Generated
\\	back slash
\`	back quote
\'	single quote
\l	line feed
\n	newline
\r	carriage return
\s	space
\t	tab
\"	double quote

Character values can be generated using the backslash character. The octal values \o0 to \o177777, Unicode values \u0000 to \uFFFF, and decimal values \d0 to \d65535 can be contained in strings to select specific characters.

Tuples

Tuples are elements delimited by square brackets. For example [1 2 3] or [a b c]. Tuples are often used to represent relationships. Relationship tuples are comprised of a literal relationship and zero or more key-value pairs (called terms) enclosed in square brackets. For example

```
[Fault :What nil :Where nil :When nil]
```

is a tuple. Resources, ephemerons, and instances can all be represented as tuples.

Premises

Premises are tuples that represent language objects.

```
> (premise (new Fault :What "Printer Error" :Where "Line 287"
                      :When "2014-003-16T21:54"))

.: [Fault :What "Printer Error" :Where "Line 287" :When "2014-003-16T21:54"]
```

For a relationship or thought, the **premise** function will retrieve a premise representation of the relationship or thought. The **uses** keyword coalesces many relations into one. Consider the following example where the **get** function retrieves the instance E_1.

```
> (relation A
    :X 0
  )

.: A

> (relation B
    :Y 1
  )

.: B

> (relation C
    :Z 0
  )

.: C

> (relation D
    uses {A B C}
  )

.: D
```

```
> (new D)

.: D_1

> (premise D_1)

.: [D ^ D_1 :X 0 :Y 1 :Z 0]

> (relation E
    uses {D}
    :Z 2
  )

.: E

> (new E)

.: E_1

> (premise E_1)

.: [E ^ E_1 :X 0 :Y 1 :Z 2]
```

Patterns

A pattern is a premise that is used to match thoughts. A pattern consists of a relation name or thought identifier list and some combination of slot bindings, slot tests, or both, enclosed within square brackets. A slot binding is a slot name, followed by **as** or **each**, followed by a symbol. A slot test is either a call containing a function that returns a Boolean value, or a slot name, followed by a predicate function name, followed by a value . Some examples of patterns are as follows:

```
    [Prediction :What ?what :Start ?start]          ; two slot bindings

    [Hypothesis ^ ?me :If ?if :Then ?then]          ; three slot bindings

    [Solution as ?s :Duration > \@t{5000}]          ; one binding one test
```

Patterns can be used in **with** function calls in order to perform rule based inference. For example, we first create a relation and some instances like this:

```
> (relation Fact :All :Are)

.: Fact

> (new Fact :All people :Are mortal)

.: Fact_1

> (new Fact :All philosophers :Are people)

.: Fact_2
```

Then the **with** expression can infer a conclusion

```
> (with [Fact :All as ?S :Are as ?M]
        [Fact :All = ?M :Are ?P)]
    get
        (knew Fact :All ?S :Are ?P))

.: [Fact ^ Fact_3 :All philosophers :Are mortal]
```

The patterns in the **with** expression above that matched the thoughts Fact_1 and Fact_2 are

```
    [Fact :All ?M :Are ?P]   matches Fact_1 and Fact_2

    [Fact ^ ?f :All ?S (= (:Are ?f) ?M)] matches Fact_2 only.
```

Finally, the **knew** function attempts to find an existing thought whose slots match the values. If none are found, then the function creates a new thought.

```
> (knew Fact All ?S :Are ?P)

.: Fact_3
```

Calls

A call is a parenthetical expression which when evaluated, will result in a value. For example, the expression (+ 1 2 3) is a call which when evaluated will result in the value 6. The first item in the expression is the left parenthesis, followed by a function literal followed by any actual parameters, followed by a right parenthesis. The function literal may be either a language intrinsic function or a user defined function. An empty call, (), returns itself.

```
(+ 1 2 3)                    (trim " abc  ")                    ()
```

Functions

Functions (also called *exonyms*) are defined using **function**.

```
> (function foo {?a ?b}
    (+ ?a ?b)
  )

.: foo
```

```
> (function bar {?a ?b}
    (* ?a ?b)
  )

.: bar
```

Functions are invoked by creating a call—wrapping the name and arguments in parentheses.

```
> (foo 10 20)

.: 30
```

```
> (bar 10 20)

.: 200
```

Parameter Lists

There are several ways define the parameters to a function

1. No parameter list.

```
(function forgetEveryone
  (each (with Person) old)
)
```

2. As a list containing zero parameters

```
(function helloWorld {}
  (tell user "Hello World")
)
```

3. As a list containing one or more required parameters

```
(function addThem {?x ?y}
  (+ ?x ?y)
)
```

4. As a list containing a variadic remainder parameter which contains all passed arguments.

```
(function addAll {& ?numbers}
  (tell user ($ the sum of ?numbers is (apply + ?numbers) \n))
)
```

5. As a list containing optional parameters.

```
(function result {+ ?y}
   (--> default ?y 0)
   (tell user ($ result = ?y \n))
)
```

6. As a list containing optional parameters with default values.

```
(function result {+ ?y := 0}
   (tell user ($ result = ?y \n)) )
```

7. As a list containing optional or required parameters with exclusive values.

```
(function result {?x == 1 + ?y == 2}
   (tell user ($ result = (+ ?x ?y) \n)) )
```

8. As a list containing required, optional, or remainder parameters.

```
(function result {?x + ?y := 1 & ?z}
  (tell user ($ result = (apply * (& ?x ?y ?z)) \n))
)
```

9. As a list containing required and optional keyword parameters.

```
(function result {?color % ?length := 0 ?width := 0 ?height := 0}
  (tell user ($$ color \s ?color , \s length \s ?length , \s width \s ?width
, \s height \s ?h) ))
)
```

10. Function parameters are automatically evaluated by default. Macro parameters are unevaluated by default.

```
(macro result {?x ?y ?z}
  (tell user ($$ (eval ?x( ", " (eval ?y) ", " (eval ?z) \n))
)
```

Parameter Specifications

Any required parameters are specified first. A failure is caused if a required parameter is not supplied with an argument when a function is called.

```
(function addThem {?x ?y}  ; ?x and ?y are required.
    "Add two numbers."
    (+ ?x ?y)
 )
```

The optional parameter sigil, the plus sign (**+**), precedes any optional ordered parameters. Optional parameters are processed sequentially and are bound to **nil** if no default values are specified.

```
 (function AddTwoToFive {?a ?b + ?c := 0 ?d := 0 ?e}
    "Add between two and five numbers."
    (apply + (& ?a ?b ?c ?d ?e)
)
```

Default values may immediately follow an optional parameter symbol. The default value sigil, the colon equal sign (**:=**), precedes a default value to a symbol. Default values may only be literals, times, numbers, or strings. The default value must not be a symbol, resource, tuple, call, or list.

```
(function addThem {+ ?x := 0 ?y := 0}; ?x and ?y both default to zero.
    "Add two numbers."
    (+ ?x ?y)
 )
```

Exclusive values may immediately follow any parameter symbol. The exclusizwe value sigil, the double equal sign (**==**), precedes an exclusive value to a symbol. Exclusive values may only be literals, times, numbers, or strings. The exclusive value must not be a symbol, resource, tuple, call, or list.

```
(function attempt {?test ?because == since ?reason}; ?because equals since
    "report if test fails."
    (unless ?test (signal failure :Name BadAttempt :Text ?resaon)
 )
```

A keyword parameter is a symbol following the keyword sigil (%). The keyword literal is matched to the arguments and the value following the literal is placed into the symbol.

```
 (macro myFor {?y % ?as  & ?body}
    (eval (` (for  , ?as in , ?y , ?body)))
 )
```

A function or macro containing required keyword parameters would be called as follows

```
(myFor {Jane Jack Joe} as ?name (tell user ($ ?name \n)))
```

Keywords are matched sequentially. Keywords follow the keywords sigil, the percent sign (**%**). Unordered keyword arguments are matched in any order they occur in the argument list.

```
(macro myStep { ?v % ?from ?to ?by := 1 & ?body}
   (eval (` (step , ?v from , ?from to , ?to by (default , ?by 1) ,_ ?body)))
)
```

A variadic parameter sigil, the ampersand (**&**), ties the remaining arguments together into a list for the subsequent symbol. Only one symbol is permitted after the ampersand. The remainder parameter will be bound to an empty list if no actual parameters are supplied.

```
(function AddTwoOrMore {?x ?y & ?z}
  (if (void-p ?z)
      (+ ?x ?y)
   else
      (apply + (& ?x ?y ?z)))
)
```

It would be called as follows

```
(myStep ?i to 100 by 2 from 1  (tell user ($ ?i \n)))
```

In functions, all arguments are evaluated before the function body is applied. This is not the case for macros. Macro parameters remain unevaluated until they are explicitly evaluated with the **eval** function within the body of the macro definition.

```
(macro callLength {?f}
   (ensure Call ?f)
   (# (eval ?f))
)
```

More examples…

```
(function foo
  "No parameter list."
  (do nothing))

(function foo {}
   "Zero length param list."
  (do nothing))

(function foo {?x ?y}
  "Fixed number of required parameters."
  (+ ?x ?y))

(function foo {+ ?x := 0 ?y := 0}
  "Optional parameters with default values."
  (+ ?x ?y))

(function foo {?w + ?x ?y & ?z}
  "Required, optional, and remaining parameters."
  (--> default ?x 0)
  (--> default ?y 0)
  (apply + (& ?w ?x ?y ?z)))

(function foo {+ ?w := 0  ?x := 0 ?y := 0 & ?z }
  "Optional and remaining parameters with default values."
  (apply + (& ?w ?x ?y ?z)))

(macro define {?name ?args & ?body}
  "Unevaluated parameters."
  (eval (` (function , ?name , ?args ,_ ?body))))
```

Auto-named Functions

Auto-named functions (also called *endonyms*) are created by omitting the function name in the definition. When the function name is omitted, the Premise interpreter will automatically generate a function name for the function.

```
> (function {?x ?y}
    (* ?x ?y)
  )

.: fn-24
```

```
> (function {& ?args}
    (apply * ?args)
  )

.: fn-25
```

```
> (function
    (with Truck ^ as ?t
       do (old ?t)))

.: fn-26
```

Anonymous Functions

Anonymous functions (i.e., also called *anonyms*) are implemented using **given**. An anonym expression can stand in for a named or auto-named function call.

```
> ((given {?x ?y} (* ?x ?y)) 2 3)          ; same effect as (fn-24 2 3)

.: 6
```

Generators

A generator is a function that creates a series. Generators are created using the **generator** function. When a generator is invoked it will produce a series.

```
> (generator generateABC {}
    (yield a)
    (yield b)
    (yield c)
    (done))

.: generateABC


> (collect ?e in (generateABC) ?e)

.: {a b c}

> (generateABC)

.: {...}
```

Macros

A macro is a call that evaluates an expanded expression. Macros are defined using **macro**. The back quote function (`) expands an expression by substituting subexpressions that follow commas, then it evaluates the resultant expression.

```
> (macro plus {?a ?b}
    (eval (` (+ , ?a , ?b)))
  )

.: plus
```

```
> (macro prod {?a ?b}
    (eval (` (* , ?a , ?b)))
  )

.: prod
```

Macros are invoked by wrapping the macro name and arguments in parentheses.

```
> (plus 10 20)

.: 30


> (prod 10 20)

.: 200
```

4. Control Flow

The following expressions permit alterations to the normal sequential flow of expression evaluation.

Conditional Expressions

Conditional expressions branch the flow of evaluation to alternate paths.

if

The if intrinsic provides branched conditional flow of evaluation.

```
> (if (= 1 2)
      (tell user ($ An impossibility.))
      (tell user ($ I'm certain. \n))
   or (= 2 1)
      (tell user ($ Another impossibility.))
      (tell user ($ Most definitely. \n))
   or (/= 1 1)
      (tell user ($ A total absurdity.))
      (tell user ($ Without a doubt. \n))
   else
      (tell user ($ The world is right as rain. \n))
      HelloWorld)

The world is right as rain.
.: HelloWorld
```

case

The case intrinsic provides branching based on a value.

```
> (case red
     of green (tell user ($ Nature's)) (tell user ($ glory. \n))
     of blue  (tell user ($ A wondrous)) (tell user ($ beauty. \n))
     in {brown black purple indigo violet} (tell user ($ A fabulous selection. \n))
   else
        (tell user ($ Hmmm... I didn't consider that one.))
        (tell user ($ ($) Bravo. \n))

Hmmm... I didn't consider that one. Bravo.
.: nil
```

on

The **on** intrinsic provides branched conditional flow of evaluation.

```
> (on (= 1 2)
    (tell user ($ A Boolean.))
    (tell user ($ An impossibility. \n)))

An impossibility
.: told

> (on (= 1 1)
    (tell user ($ Without a doubt. \n))
    (tell user ($ A total absurdity.)))

Without a doubt.
.: told
```

do

The **do** intrinsic provides subroutine resumption.

```
> (do
      (escape)
      (never-happens)
   resume
     (tell user ($ Resumed execution \n))
   finally
    (tell user ($ Done. \n)))

Resumed execution
Done.
.: told
```

try

The **try** intrinsic provides exception handling,

```
> (try
    (signal Failure :Name MyFault :Text "Failed Miserably")
   learn ?failure
    (tell user ($ Handled ?failure \n))
   finally
    (tell user ($ Done. \n)))

Handled [Failure :Name MyFault :Text "Failed Miserably"]
Done.
.: told
```

Looping Expressions

Certain expressions are used to loop through a sequence or iterate over a series of values.

loop

The **loop** intrinsic allows conditional repetition of one or more expressions.

```
> (global ?I 100)

.: true

> (loop (--> -- ?I) until (= ?I 90))

.: 90

> (loop (--> ++ ?I) while (< ?I 100))

.: 100

> (loop (--> -- ?i) repeat 100)

.: 0
```

for

The **for** intrinsic repeatedly binds one or more variables through elements of a sequence.

```
> (for ?x in {hello world} (tell user ($$ ?x \s)))

hello world .: told

> (for ?x in "hello world" (tell user ($$ ?x \s)))

h e l l o   w o r l d .: told

> (for ?k ?v in {a 1 b 2 c 3 d 4}
    (tell user ($ ?k ($$ ?v ,))))

a 1, b 2, c 3, d 4, .: told

> (for ?x ?y ?z per {{a 1 $}{b 2 @}{c 3 #}{d 4 /}}
    (tell user ($ ?x ?y ($$ ?z ,))))

a 1 $, b 2 @, c 3 #, d 4 /, .: told
```

step

The **step** intrinsic increments or decrements a symbol over a series of numeric values.

```
> (step ?i from 5 to 9
    (tell user ($$ "?i = " ?i ", " )))

?i = 5, ?i = 6, ?i = 7, ?i = 8, ?i = 9, .: told

> (step ?i from 100 to 50 by -10
    (tell user ($$ "?i = " ?i ", " )))

?i = 100, ?i = 90, ?i = 80, ?i = 70, ?i = 60, ?i = 50, .: told

> (step ?i from 0 to 0.5 by 0.1
    (tell user ($$ "?i = " ?i ", " )))

?i = 0, ?i = 0.1, ?i = 0.2, ?i = 0.3, ?i = 0.4, ?i = 0.5, .: told
```

repeat

The **repeat** intrinsic allows repetition of one or more expressions for a specified number of iterations.

```
> (repeat 1000 (--> ++ ?i))

.: 1100

> (repeat 5 (tell user ($$ hi \s))

hi hi hi hi hi .: told
```

count

The **count** intrinsic repeatedly binds one or more variables to elements of a sequence and returns the count..

```
> (count ?x in {the quick brown fox} as ?total
    (tell user ($ ?x \t)))

the     quick   brown   fox     .: 4
```

while

The **while** intrinsic allows repetition of one or more expressions while a condition is yes.

```
> (global ?N 0)

.: yes

> (while (< ?N 5)
    (tell user ($ ?N \n))
    (--> ++ ?N))

0
1
2
3
4
.: 5
```

until

The **until** intrinsic provides repetition of one or more expressions until a condition is yes.

```
> (global ?S {a b c d 1 2 3 4} ?i 0 ?result {})

.: true

> (until (number-p (@ ?S (--> ++ ?i)))  ; until S[++?i] is a number
    (--> & ?result {{(@ ?S ?i) bar}}))  ;   merge ?result with { S[?i] bar }

.: {{a bar}{b bar}{c bar}{d bar}}
```

Control Flow Expressions

Control flow expressions redirect the flow of control.

confirm

The **confirm** intrinsic signals a failure if the test fails.

```
> (function multiply {?a ?b}
    (confirm (taxon-p ?a number) since ($ ?a must be a number))
    (confirm (taxon-p ?b number) since ($ ?b must be a number))
    (* ?a ?b))

.: multiply

> (multiply  10 banana)

.: [Failure :Name DataTypeFailure :Text "banana must be a number"]
```

confute

The **confute** intrinsic signals a failure if the test succeeds.

```
> (confute (= red blue) since "colors must be different")

.: no

> (function reducing {?sequence ?function}
    (ensure sequence ?sequence function ?function)
    (confute (<= (# ?sequence) 1) since "Two or more elements are required.")
    (let ?result (@ ?sequence))
    (for ?element in (rest ?sequence) (--> ?function ?result ?element)))

.: reducing

> (reducing {1} +)

.: [Failure :Name Confutation :Text "Two or more elements are required."]
```

break

The **break** intrinsic transfers control out of the containing loop.

```
> (do (step ?i from 1 to 10
        (if (= ?i 3) (break nil))
        (tell user ($ ?i \n)))
      (tell user ($ done \n)))

1
2
done
.: nil
```

continue

The **continue** intrinsic transfers control to the next iteration of the containing loop.

```
   > (for ?letter in "Premise"
       (if (in "meis" ?letter) (continue))
       (tell user ($ Letter is ($$ ?letter .) \n)))

   Letter is P.
   Letter is r.
   .: nil
```

exit

The **exit** intrinsic transfers control out of the current task. Whereas return may exit from a nested function, exit terminates the task. Exit can be thought of as a task level return.

```
> (concurrent
    (step ?x from 1 to 999999999
      (if (> ?x 100) (exit exited))))

.: {Task-1}

> (await task-1)

.: exited

> (free task-1)

.: freed
```

signal

The **signal** intrinsic transfers control to an encompassing try expression.

```
> (function myFun
    (try
      (signal Failure :Name BadAttitude :Text "example")
     learn ?f
      (case (:Name ?f)
         of BadAttitude
            (tell user ($ Caught inside myFun. \n))
            (signal ?f)))
  )

.: myFun

> (function main {& ?args}
    (try
        (myFun)
     learn ?f
      (case (:Name ?f)
         of BadAttitude
            (tell user ($ Caught inside main. \n))))
  )

.: main

> (main)

Caught inside myFun.
Caught inside main.
.: nil
```

return

The **return** intrinsic transfers control out of the current function.

```
> (function a {?i} (if (= ?i 5) (return nil)) (tell user ($ ?i \n)))

.: a

> (function b {?i} (if (> ?i 4) (return 4 from b)) (tell user ($ ?i \n)) (return 2))

.: b

> (function c {?k ?v} (return {?k ?v} from function))

.: c

> (function d (a 30))

.: d

> (do
     (a 5)
     (a 1)
     (let ?i (b 5))
     (tell user ($ ?i \n))
     (tell user ($ (@ (c 100 90) 2) \n))
     (d))

1
4
90
30
.: nil
```

ensure

The **ensure** intrinsic transfers control to an encompassing try expression if a type check fails.

```
> (function multiply {?a ?b}
    (ensure number ?a number ?b)
    (* ?a ?b))

.: multiply

> (multiply 10 banana)

.: [Failure :Name EnsureType :Text "The value banana must be a number"]

> (multiply  10 5)

.: 50
```

escape

The **escape** function transfers control to an encompassing **do** expression containing a **resume** tag.

```
> (do
    (escape)
    (tell user "Never executed")
   resume here
     (tell user ($ Resumed execution \n))
   finally
    (tell user ($ Done. \n)))


Resumed execution
Done.
```

go

The **go** intrinsic transfers control to a function.

```
> (relation MyQueue   :Items {} :Capacity 10
                    !count (given {?me}(# (:Items ?me)))
                    !full-p (given {?me}(>= (!count ?me) (:Capacity ?me))))

.: Queue

> (function produce_or_consume {?q}

    (here produce)

    (tell user ($ producing \n))
    (if (not (!full-p ?q))
      (repeat (random 1 to (- (:Capacity ?q) (!count ?q)))
       (enq ?q :Queue each (new Item))))
    (go consume)

    (here consume)

    (tell user ($ consuming \n))
    (if (more-p (:Queue ?q))
      (repeat (random 1 to (# (:Queue ?q)))
         (deliver (pop (:Queue ?q)))))
    (on (void-p (:Queue ?q))
        done
        (go produce))
)

.: consume

> (produce_or_consume (new MyQueue))

producing
consuming
producing
consuming
producing
consuming
producing
consuming
producing
consuming
producing
consuming
producing
consuming
producing
consuming
producing
consuming
producing
consuming
producing
consuming
.: done
```

Multitasking Expressions

The Premise Language facilitates asynchronous and concurrent expression evaluation.

concurrent

The **concurrent** function runs expressions asynchronously and returns immediately.

```
> (concurrent (+ 1 2 3 4) (/ 1000 57))

.: {task-1 task-2}

> (await task-1)

.: 10

> (await task-2)

.: 17.54385964912281

> (free {task-1 task-2})

.: nil
```

complete

The **complete** function executes expressions concurrently and returns after all have completed.

```
> (complete (step ?i from 0 to 50000000 (do nothing))
      (step ?i from 1 to 10000000 (do nada)))

.: {Task-1 Task-2}

> (await task-1)

.: nothing

> (await task-2)

.: nada

> (free {task-1 task-2})

.: freed
```

scatter

The **scatter** function applies functions in parallel and returns a list of results.

```
> (function double {?x} (* ?x 2))

.: double

> (scatter {5} {++ -- double})

.: {6 4 10}

> (scatter {{the quick brown fox jumped over the lazy dogs}}
           {except rest top last})

.: {{the quick brown fox jumped over the lazy }
    {quick brown fox jumped over the lazy dogs}
    {the}
    {dogs}}

> (scatter {2 3 4} {+ * -})

.: {9 24 -5}
```

5. Code Examples

The following are simple examples demonstrating the language.

Inference

Logical pattern matching and declarative inference.

```
  (relation Statement
    :All
    :Are
  )

  (new Statement :All people :Are mortal)

  (new Statement :All philosophers :Are people)

  (with [Statement :All as ?s :Are as ?m]
        [Statement :All = ?m :Are as ?p]
   list
        (knew Statement :All ?s :Are ?p))
```

```
> (relation Statement
    :All
    :Are
  )

.: Statement

> (new Statement :All people :Are mortal)

.: Statement_1

> (new Statement :All philosophers :Are people)

.: Statement_2

> (with [Statement :All as ?s :Are as ?m]
        [Statement :All = ?m :Are as ?p]
   list
        (knew Statement :All ?s :Are ?p))

.: {Statement_3}

> (with Statement)

.: {[Statement ^ Statement_1 :All people :Are mortal]
    [Statement ^ Statement_2 :All philosophers :Are people]
    [Statement ^ Statement_3 :All philosophers :Are mortal]}
```

Memoization

Memoization is the process of saving intermediate results for lookup rather than recomputation. Once a value has been computed, it can be memorized so that if the value is needed again, it can be easily retrieved rather than recomputed. The following example uses memoization in computing Fibonacci numbers.

```
> (relation Memoized
    :Number
    :Result
  )

.: Memoized

> (new Memoized :Number 0 :Result 0)

.: Memoized_1

> (new Memoized :Number 1 :Result 1)

.: Memoized_2

> (function fib {?n}
    (confirm (>= ?n 0) since
             ($ ?n must be zero or more.))
   (with
     [Memoized ^ :Number as ?n :Result as ?r] ; if a matching instance exists
    do
     (return ?r))         ; then return, otherwise do nothing
   (let ?result (+ (fib (- ?n 1)) (fib (- ?n 2))))
   (new Memoized :Number ?n :Result ?result)
   (return ?result)
  )

.: fib

> (fib 7)

.: 13

> (with Memoized)

.: {[Memoized ^ Memoized_1 :Number 0 :Result 0]
    [Memoized ^ Memoized_2 :Number 1 :Result 1]
    [Memoized ^ Memoized_4 :Number 3 :Result 2]
    [Memoized ^ Memoized_6 :Number 5 :Result 5]
    [Memoized ^ Memoized_3 :Number 2 :Result 1]
    [Memoized ^ Memoized_8 :Number 7 :Result 13]
    [Memoized ^ Memoized_7 :Number 6 :Result 8]
    [Memoized ^ Memoized_5 :Number 4 :Result 3]}
```

Event driven programming.

```
> (structure Event
    :Topic
    :Content
  )

.: Event

> (relation Fault
    :Who
    :What
    :Where
    :When
  )

.: Fault

> (function dispatcher {?message}
    (try
        (let {?taken ?next} (parse ?message)
             ?event (on (takeable ?taken) (@ ?taken) nil)

        (ensure Event ?event)

        (let ?event :Topic as ?topic :Content as ?content)
        (case ?topic
           of Percept  (onPerceptReceived ?content)
           of Ping     (onPingReceived    ?content)
           else
               (signal Failure :Name BadTopic :Text ($ Unknown topic ?topic)))
        (tell user ($ Handled ?message))
     learn ?failure
        (new Fault :Who Dispatcher :What ?failure  :When (date)
                   :Where {dispatcher ?message}))
  )

.: dispatcher

> (global ?URL (config "/application/eventUrlAndPort"))

.: true

> (service nil ?URL dispatcher) ; creates a message loop

.: Service-1

> (tell ?URL ($ (new Event :Topic Ping :Content {:Sent (moment)})))))

.: told
Handled [Event :Topic Ping :Content {:Sent \@m{201804991265124}]

> (free Service-1 cancel yes)

.: freed
```

Similarity Based Matching

The **~** and **best** language intrinsics perform similarity based comparisons to support case-based reasoning.

```
> (~ {a b c} {a d e})

.: 0.2

> (~ "a b c" "a d e")

.: 0.090909091

> (~ 57 890)

.: 0.00119904076738

> (best 2001 (range 1 to 1000000) 5)

.: {{2001 1} {2000 0.5} {2002 0.5} {1999 0.33333333333333333}
{2003 0.33333333333333333}}

> (best "this" {"that" "hat" "hit" "with" "isthmus"} 3)

.: {{"that" 0.11111111111111111} {"hat" 0.08333333333333333}
{"hit" 0.08333333333333333}}
```

Asynchrony and Concurrency

The **concurrent**, **complete**, and **task** functions run expressions asynchronously, concurrently, or as required. The concurrence function **complete** returns after all tasks have completed whereas the asynchronicity function **concurrent** returns immediately after invocation. The **task** function creates a task for deferred execution. The functions each return a list of task handles.

```
> (concurrent (step ?i from 1 to 30000000 (do something))    ; runs asynchronously
          (step ?j from 1 to 60000000 (do something-else)))

.: {Task-1 Task-2}

> (ready-p task-1)

.: no

> (await task-1)

.: something

> (await task-2 1000 timed-out) ; adding timeout and default args

.: timed-out
```

```
> (await task-2)

.: something-else

> (free {task-1 task-2})

.: freed

> (complete (step ?i from 1 to  30000000 (do someOtherThing)) ; runs concurrently
      (step ?j from 1 to  60000000 (do anotherThing)
      (step ?k from 1 to 100000000 (do oneLastThing))

.: {Task-3 Task-4 Task-5}

> (cancel task-5)

.: cancelled

> (await task-3 0 timed-out)

.: someOtherThing

> (await task-4 0 timed-out)

.: anotherThing

> (free {task-3 task-4})

.: freed

> (function sayHello {}
    (for ?c in "hello " (tell user ($ ?c))))

.: sayHello

> (global ?Tasks (task (sayHello) (sayHello) (sayHello) (sayHello)))

.: true

> (free (complete ?Tasks))

hhehelelohl lello l ol o  .: freed

> (function sayHello {}
    (critical printing
      (for ?c in "hello " (tell user ($ ?c)))))

.: sayHello

> (set ?Tasks (task (sayHello) (sayHello) (sayHello) (sayHello)))

.: true

> (free (complete ?Tasks))

hello hello hello hello .: freed

> (undeclare ?Tasks)

.: undeclared
```

Thoughts

Thoughts are tuples which reside in a knowledge base. Thoughts are created with **new** function and are destroyed with the **old** function. The **premise** function can be used to retrieve a thought. The **zap** function sets all slots of a thought to **nil**.

```
> (relation Foo :Bar :Baz)

.: Foo

> (new Foo :Bar 100)

.: Foo_1

> (premise Foo_1)

.: [Foo ^ Foo_1 :Bar 100 :Baz nil]

> ?bar

.: [Failure :Name UndefinedSymbol :Text "Symbol ?bar is undefined."]

> (bind (premise Foo_1) :Bar ?bar)

.: true

> ?bar

.: 100

> (put Foo_1 :Baz "The quick brown fox")

.: Foo_1

> (zap Foo_1 :Bar)

.: Foo_1

> (premise Foo_1)

.: [Foo ^ Foo_1 :Bar nil :Baz "The quick brown fox"]

> (zap Foo_1)

.: Foo_1

> (premise Foo_1)

.: [Foo ^ Foo_1 :Bar nil :Baz nil]
```

The **knew** function retrieves an existing thought, or if it does not exist, creates a new thought. The **known** function retrieves an existing thought given a premise. The posit function creates a new thought.

```
> (premise Foo_2)

.: [Foo ^ Foo_2 :Bar 100 :Baz nil]

> (premise Foo_3)

.: [Foo ^ Foo_3 :Bar 200 :Baz nil]

> (knew Foo :Bar 100)

.: Foo_2

> (knew Foo :Bar 200)

.: Foo_3

> (knew Foo :Bar 100)

.: Foo_2

> (known [Foo :Bar 100] )

.: Foo_2

> (known [Foo :Bar 100] )

.: Foo_2

> (known [Foo :Bar 200] )

.: Foo_3

> (known [Foo :Bar 400] )

.: nil

> (posit [Foo :Bar 400] )

.: {Foo_4}

> (known [Foo :Bar 400] )

.: Foo_4
```

The **assert** and **assume** functions create a thought containing nested thoughts. When either function encounters a nested premise, it determines whether or not the nested premise has a defined relation. If so, the assert function performs a **new** on the nested premise while the assume function performs a **knew** on the nested premise. In the example below, an Operation has a :Context property containing an anonymous function for a state that returns a list of bindings if there is a match, otherwise it returns **nil**. The :Effects property contains an anonymous function that creates and edits a successor state.

```
> (relation Operation :Name :Parameters :Context :Effects !bindings !successor)

.: Operation

> (relation Is :Type :Item)

.: Is

> (relation On :Above :Below)

.: On

> (relation Has :What :Item)

.: Has

> (assume Operation
      :Name PickUp
      :Parameters {$ob}
      :Context  { [On :Above CLEAR :Below $ob]
                  [On :Above $ob :Below TABLE]
                  [Has :What ARM :Item EMPTY]}
      :Effects  { :Affirm { [Has :What ARM :Item $ob] }
                  :Remove { [On :Above CLEAR :Below $ob]
                            [On :Above $ob :Below TABLE]
                            [Has :What ARM :Item EMPTY] }} )
.: Operation_1

> (premise Operation_1)

.: [Operation ^ Operation_1 :Name PickUp :Parameters {$ob} :Mappings {$ob ?ob}
:Context {On_1 On_2 Has_1 } :Effects {:Affirm {Has_2} :Remove  On_1 On_2 Has_1}}]

> (assert Operation
      :Name PickUp
      :Parameters {$ob}
      :Context { [On :Above CLEAR :Below $ob]
                 [On :Above $ob :Below TABLE]
                 [Has :What ARM :Item EMPTY] }
      :Effects { :Affirm { [Has :What ARM :Item $ob] }
                 :Remove { [On :Above CLEAR :Below $ob]
                           [On :Above $ob :Below TABLE]
                           [Has :What ARM :Item EMPTY] }} )


.: Operation_2

> (premise Operation_2)

.: [Operation ^ Operation_2 :Name PickUp :Parameters {$ob} :Mappings {$ob ?ob}
:Context {and On_3 On_4 Has_2 } :Effects {:Affirm {Has_2} :Remove {On_5 On_6 Has_3}}]
```

Functions

Functions are defined using the function intrinsic. Function names may not be preceded by a question mark. Functions may have no parameter list, an empty parameter list, or a list containing, required, optional, remainder or keyword parameters.

```
> (function factorial {?n}              ; named function with a parameter list
    (reduce (range 1 to ?n) *)
  )

.: factorial

> (factorial 4)

.: 24

> (function prime-p {?n}      ; named function with required parameter
    (if (< ?n 2)
        (return no)
     else
        (step ?i from 1 to (++ (ceiling (root ?n)))
          (if (and (> ?i 1)(< ?i ?n)(zero-p (% ($ ?n ?i))))
              (return no))))
    yes
  )

.: prime-p

> (prime-p 2)

.: yes

> (prime-p 6)

.: no

> (function sum {& ?args}      ; named function with a remaining parameter symbol
    returns number             ; with return type restriction
    (apply + ?args)
  )

.: sum

> (sum 1 2 3 4)

.: 10
```

```
> (function addTwoToFive {?1 ?2 + ?3 := 0 ?4 := 0 ?5 := 0} ; named function with
optional
    (apply + (& ?1 ?2 ?3 ?4 ?5))                          ; defaulted parameters
  )

.: addTwoToFive

> (addTwoToFive 5 5)

.: 10

> (addTwoToFive 5 6 7 8)

.: 26

> (function addAll {?1 ?2 + ?3 := 0 ?4 := 0 & ?rest} ; named function with optional
and
   (apply + (& ?1 ?2 ?3 ?4 ?rest))                  ; remainder parameters
  )

.: addAll

> (addAll 1 2 3 4 5 6 7 8 9 10)

.: 55

> (function forgetIt {?relation}        ; named function
    (confirm (relation-p ?prototye) since ($ ?relation must be a relation.))
    (for ?x in (with ?relation) (old ?x))
    forgotten
  )

.: forgetIt

> (with Statement)

.: {[Statement ^ Statement_1 :All people :Are mortal]
    [Statement ^ Statement_2 :All philosophers :Are people]
    [Statement ^ Statement_3 :All philosophers :Are mortal]}

> (forgetIt Statement)

.: forgotten

> (with Statement)

.: {}

> (function {& ?args}         ; auto-named function with
     (apply * ?args)          ; remaining parameter symbol
  )

.: fn-1
```

```
> (fn-1 1 2 3 4)

.: 24

> (function transitivity               ; no parameters
    (with [Statement :All as ?a :Are as ?b]
          [Statement :All = ?b :Are as ?c]
       do
          (knew Statement :All ?a :Are ?c))
  )

.: transitivity


> (macro myFor {?x {in ?y} & ?body}    ; a required keyword parameter
    (eval (` (apply for (& , ?x in , ?y   ,_ ?body))))
  )

.: myFor

> (myFor ?name in {Jane John Sally} (tell user ($ ?name \n)))

Jane
John
Sally

.: told

> (macro myStep {?v {from ?a} {to ?z} + {by ?i = 1} & ?body} ; optional keyword
    (eval (` (step , ?v from , ?a to , ?z by , ?i ,_ ?body))))
  )

.: myStep

> (myStep ?x from 1 to 5 by 2 (tell user ($ ?x \n)))

1
3
5

.: told


> (myStep ?x from 1 to 5 (tell user ($ ?x \n)))

1
2
3
4
5

.: told
```

Delegates

Delegates are functions that are passed as parameters to other functions.

```
> (relation Product  :Name  :Price )

.: Product

> (relation Cart  :Items  {}  !total  Cart_Total )

.: Cart

> (function Cart_Total {?me ?SubTotDelegate ?TotalDelegate ?DiscountDelegate}
    (ensure Cart ?me Function ?SubTotDelegate Function ?TotalDelegate
            Function ?DiscountDelegate)
    (let ?subTotal (sum (map (:Items ?me) :Price)))
    (?SubTotDelegate ?subTotal)
    (?DiscountDelegate ($ Discounts applied.\n))
    (?TotalDelegate (:Items ?me) ?subTotal))

.: Cart_Total

> (relation Program  :Cart  !onNew (given {?me}(put ?me :Cart (new Cart))))

.: Program

> (function main {& ?args}
    (let ?me (new Program))

    (enq (:Cart ?me) :Items (new Product :Name Cheese :Price 5.50))
    (enq (:Cart ?me) :Items (new Product :Name Bread :Price 7.00))
    (enq (:Cart ?me) :Items (new Product :Name Milk :Price 4.65))
    (enq (:Cart ?me) :Items (new Product :Name Eggs :Price 5.95))

    (tell user ($ Your total is
                (!total (:Cart ?me)
                        (given {?sub}(tell user ($ The subtotal is ?sub \n)))
                        (given {?products ?sub}
                         (if (> (# ?products) 3)
                             (* ?sub 0.90)
                          else ?sub))
                        (given {?msg}(tell user ?msg)))
                \n \n)))

.: main

> (main)

The subtotal is 23.10
Discounts Applied.
Your total is 20.7

.: told
```

Poker Hand Analyzer

This function tells what kind of poker hand each player has.

```
> (enumeration Suits
    Clubs   Diamonds   Hearts   Spades
  )

.: Enumeration-1

> (enumeration Ranks
    Ace    Two   Three  Four  Five   Six   Seven
    Eight  Nine  Ten    Jack  Queen  King  Joker
  )

.: Enumeration-2

> (relation Card
    :Rank
    :Suit
    :Dealt
  )

.: Card

> (relation Player
    :Order
    :Cards {}
    :Hand  Unknown
  )

.: Player

> (function setup
    (repeat 2   ; use 2 decks
      (for ?suit ?i per (entries  Suits)
        (for ?rank ?j per (entries  Ranks)
          (if (/= ?rank Joker) (new Card :Rank ?rank :Suit ?suit)))))
    (let ?order 0)
    (repeat (random 2 to 8) (new Player :Order (--> ++ ?order)))
  )

.: setup

> (function deal
    (with Card ^ as ?card do (bind ?card :Dealt no))
    (with Player ^ as ?player do (bind ?player :Cards {} :Hand unknown))
    (let ?players (which Player sort :Order < )
            ?cards   (shuffle (which Card)))
    (repeat (config "/Game/Poker/CardsPerPlayer")
      (for ?player in ?players
        (so {?card (pop ?cards)}
          (enq ?player :Cards ?card)
          (put ?card :Dealt yes)))
    )
   dealt
  )

.: deal
```

```
> (enumeration Hands
    Nothing
    OnePair
    TwoPairs
    ThreeOfAKind
    Straight
    Flush
    FullHouse
    FourOfAKind
    StraightFlush
  )

.: Enumeration-3

> (function analyze
    (with [Player ^ as ?p :Cards as ?cards :Hand = Unknown]
          [Card ^ as ?c1 (contains ?cards ?c1) :Suit as ?s :Rank as ?r]
          (let ?n (position Ranks ?r))
          [Card ^ as ?c2 (contains ?cards ?c2) :Suit = ?s :Rank = (@ Rank (+ ?n 1))]
          [Card ^ as ?c3 (contains ?cards ?c3) :Suit = ?s :Rank = (@ Rank (+ ?n 2))]
          [Card ^ as ?c4 (contains ?cards ?c4) :Suit = ?s :Rank = (@ Rank (+ ?n 3))]
          [Card ^ as ?c5 (contains ?cards ?c5) :Suit = ?s :Rank = (@ Rank (+ ?n 4))]
          (/= ?c1 ?c2 ?c3 ?c4 ?c5)
       do
          (put ?p :Hand StraightFlush))

    (with [Player ^ as ?p :Cards as ?cards :Hand = Unknown]
          [Card ^ as ?c1 (contains ?cards ?c1) :Rank = ?r]
          [Card ^ as ?c2 (contains ?cards ?c2) :Rank = ?r]
          [Card ^ as ?c3 (contains ?cards ?c3) :Rank = ?r]
          [Card ^ as ?c4 (contains ?cards ?c4) :Rank = ?r]
          (/= ?c1 ?c2 ?c3 ?c4)
       do
          (put ?p :Hand FourOfAKind))

    (with [Player ^ as ?p :Cards as ?cards :Hand = Unknown]
          [Card ^ as ?c1 (contains ?cards ?c1) :Rank as ?m]
          [Card ^ as ?c2 (contains ?cards ?c2) :Rank = ?m]
          [Card ^ as ?c3 (contains ?cards ?c3) :Rank = ?m]
          [Card ^ as ?c4 (contains ?cards ?c4) :Rank as ?n (/= ?m ?n)]
          [Card ^ as ?c5 (contains ?cards ?c5) :Rank = ?n]
          (/= ?c1 ?c2 ?c3 ?c4 ?c5)
       do
          (put ?p :Hand FullHouse))

    (with [Player ^ as ?p :Cards as ?cards :Hand = Unknown]
          [Card ^ as ?c1 (contains ?cards ?c1) :Suit as ?s]
          [Card ^ as ?c2 (contains ?cards ?c2) :Suit = ?s]
          [Card ^ as ?c3 (contains ?cards ?c3) :Suit = ?s]
          [Card ^ as ?c4 (contains ?cards ?c4) :Suit = ?s]
          [Card ^ as ?c5 (contains ?cards ?c5) :Suit = ?s]
          (/= ?c1 ?c2 ?c3 ?c4 ?c5)
       do
          (put ?p :Hand Flush))

    (with [Player ^ as ?p  Cards ?cards :Hand = Unknown]
          [Card ^ as ?c1 (contains ?cards ?c1) :Rank as ?r]
          (let ?n (position Ranks ?r))
          [Card ^ as ?c2 (contains ?cards ?c2) :Rank = (@ Ranks (+ ?n 1))]
          [Card ^ as ?c3 (contains ?cards ?c3) :Rank = (@ Ranks (+ ?n 2))]
          [Card ^ as ?c4 (contains ?cards ?c4) :Rank = (@ Ranks (+ ?n 3))]
          [Card ^ as ?c5 (contains ?cards ?c5) :Rank = (@ Ranks (+ ?n 4))]
```

```
          (/= ?c1 ?c2 ?c3 ?c4 ?c5)
       do
          (put ?p :Hand Straight))

    (with [Player ^ as ?p :Cards as ?cards :Hand = Unknown]
          [Card ^ as ?c1 (contains ?cards ?c1) :Rank as ?n]
          [Card ^ as ?c2 (contains ?cards ?c2) :Rank = ?n]
          [Card ^ as ?c3 (contains ?cards ?c3) :Rank = ?n]
          (/= ?c1 ?c2 ?c3)
       do
          (put ?p :Hand ThreeOfAKind))

    (with [Player ^ as ?p :Cards as ?cards :Hand = Unknown]
          [Card ^ as ?c1 (contains ?cards ?c1) :Rank as ?m]
          [Card ^ as ?c2 (contains ?cards ?c2) :Rank = ?m]
          [Card ^ as ?c3 (contains ?cards ?c3) :Rank as ?n (/= ?m ?n))]
          [Card ^ as ?c4 (contains ?cards ?c4) :Rank = ?n]
          (/= ?c1 ?c2 ?c3 ?c4)
       do
          (put ?p :Hand TwoPairs))

    (with [Player ^ as ?p :Cards as ?cards :Hand = Unknown]
          [Card ^ as ?c1 (contains ?cards ?c1) :Rank as ?m]
          [Card ^ as ?c2 (contains ?cards ?c2) :Rank = ?m]
          (/= ?c1 ?c2)
       do
          (put ?p :Hand OnePair))

    (with [Player ^ as ?p :Cards as ?cards :Hand = Unknown]
       do
          (put ?p :Hand Nothing))
    ready
  )

.: analyze

> (do (setup) (deal) (analyze))

.: ready
```

Theories

In the Premise language, a theory is a file with a **.th** or **.theory** extension. There are two types of theory files, **module** theories and **domain** theories.

Modules

Modules are literals that encapsulate function definitions. In practice, a function has a module preceding the function name, (as in “module.function”). Functions having a module in their literal are “qualified”. Functions without modules are “unqualified”. The module intrinsic creates modules. The extend intrinsic allows more function definitions to be added to a module. The using intrinsic sets or returns the current module. The require intrinsic allows functions to be visible between modules. Module theories typically contain function definitions.

```
;;; SuperMath.Theory

(module SuperMath

  (facility power {?x + ?y := 1}        ; private to the module
    (** ?x ?y)
  )

  (function superpower {& ?numbers}     ; public to the module
    (if (void-p ?numbers)
        (return \%unknown))
    (let ?result (@ ?numbers))
    (for ?n in (rest ?numbers)
      (--> power ?result ?n))
    ?result
  )
)
```

```
> (modules)               ; view the modules collection

.: {Apex Base IO KB Math Tasks User}

> (using User)                    ; work in a specific module

.: User

> (module Arithmetic              ; define a new module Arithmetic
    (function sum {& ?args}
      (apply + ?args)
    )
  )

.: Arithmetic

> (modules)

.: {Arithmetic Apex Base IO KB Math Tasks User}

> (sum 1 2 3 4)              ; is sum visible from user module?

.: [Failure :Name ArgumentValue :Text "The function sum isn't found"]

> (Arithmetic.sum 1 2 3 4)         ; is fully qualified function visible?

.: 10

> (require Arithmetic as a)   ; user module requires definitions

.: Arithmetic

> (sum 1 2 3 4) ; now sum is visible

.: 10

> (a.sum 1 2 3 4) ; now a.sum is visible

.: 10

> (Arithmetic.sum 1 2 3 4) ; still accessible as Arithmetic.sum

.: 10

> (extend Arithmitic

     (function product {& ?args}      ; add a new function
       (apply * ?args)
     )
  )

.: Arithmetic
```

Domains

A domain, also called a problem space or realm, is an environment containing relationships, rules, and references to prerequisite domains. The **domain** function creates a domain. Domain theories contain problem space definitions.

```
;;; Basic-Arithmetic

(domain Basic-Arithmetic
     (relation Summation :Result 0)
     (relation Addend :Value 0 :Status pending))
```

```
;;; Arithmetic.Theory

(domain Arithmetic

  (require Basic-Arithmetic from "file:///basic-arith.theory" )  ; a prerequisite

  (rule Basic-Addition
    salience 100
    problem as ?problem state as ?state
    within ?state
      :Facts
        [Summation ^ as ?sum :Result as ?result]
        [Addend ^ as ?add :Value as ?value :Status = pending]
    do
      (put ?sum :Result (+ ?value (default ?result 0)))
      (put ?add :Status added))

  (rule Report
    salience 0
    problem as ?p  state as ?state
    within ?state
      :Facts
        [Summation as ?sum :Result as ?result]
        [Addend ^ as ?add :Status as ?status]
        (notany [Addend = ?add :Status = pending])
    do
      (put ?p :Result (format "For ~s, the result is ~s." ?p ?result)))

  (rule Clean-up
    salience -100
    with [Addend ^ as ?a :Status = added]
    do   (old ?a))
)
```

Rules

A rule is a task that operates when its preconditions are satisfied. The **rule** function creates a rule, which can be either causal (forward chaining) or logical (backward chaining).

```
(rule name
  memo description      ; the purpose of the rule  as a string
  domain realm          ; the rule package containing this rule
  salience 0            ; the relative priority of this rule
  problem as symbol     ; the a symbol for binding the current problem
  state as symbol       ; the a symbol for binding the current state
  match                 ; the match to perform, either: each, with, or within
     expression ...     ; the premises that generate symbol bindings
  do expression ...     ; the expressions to evaluate
  then next             ; the next domain to use in solving the problem.
)
```

Memo

A string description of the rule's purpose.

```
    memo "Calculate all credits and debits"
```

Domain

The domain specifies the ruleset of the rule. If omitted the the encompassing domain, or **nil**. Is used.

```
    domain Arithmetic
```

Salience

The salience is the relative priority of the rule. It is a 64 bit integer. If omitted the default is zero (0).

```
    salience 100
```

Problem

The problem identifies the specific transaction for which the rule is a candidate.

```
problem as ?problem
```

State

The state identifies the specific situation to be addressed by the rule.

```
state as ?state
```

Match

This section identifies premises which must be matched to the knowledge base. . A **with** match uses whatever expressions are provided.. An **each** match iterates a symbol over a container. (An each match may follow a with match within a rule.) A **within** match uses a thought and the slots found within the thought to find matches

```
with
  [On :Above BLUE :Below TABLE] ...

each ?item in ?list
  [On :Above = ?item :Below = TABLE] ...

within ?state
  :Facts [On :Above BLUE :Below TABLE] ...
  :Goals [On :Above BLUE :Below TABLE] ...
```

Do

This section identifies expressions which must be evaluated if the rule is selected for execution.

```
do (deq ?state :Facts (known [Has :What ARM :Item ?item]))
   (enq ?state :Facts (known [HAS :What ARM :IteM RED]))
```

Then

This section identifies the next domain (ruleset) to use for the continued pursuit of this problem.

```
 Then  FINISHED  ; a domain with no rules defined.
```

Thus, some sample rules are as follows.

```
(rule Basic-Addition
  memo "Add a bunch of numbers."
  salience 100
  problem as ?problem
  state as ?state
  within ?state
     :Facts
       [Summation ^ as ?sum :Result as ?result]
       [Addend ^ as ?add :Value as ?value :Status = pending]
  do
     (put ?sum :Result (+ ?value (default ?result 0)))
     (put ?add :Status added))


(rule Report
  memo "Report the result."
  salience 0
  problem as ?problem
  state as ?state
  within ?state
    :Facts
       [Summation as ?sum :Result as ?result]
       [Addend ^ as ?add :Status as ?status]
       (notany [Addend = ?add :Status = pending])
  do
     (put ?p :Result (format "For ~s, the result is ~s." ?p ?result)))
```

Problems

A **problem** is a gap, obstacle, novelty, or process. A problem state describes the situation to be addressed. A state may be a gap, lacuna, obstacle, novelty, or process specified using premises. The rules implementation should address how to form a solution for the situation.

```
(problem  name
  memo     description
  domain   realm
  start    time
  until    time
  state    state
  status   open (default) | idle | busy | wait | fail | done
  result   nil
)
```

For example the following problem relates to the Arithmetic domain.

```
  (problem  SimpleAddition
     domain Arithmetic
     state (assume Gap
              :Facts [Addend :Value 5]
                     [Addend :Value 7]
                     [Addend :Value 11]
                     [Summation] )))

  (think)
```

The above expressions will create the following premise in the knowledge base and run any rules.

```
[Problem ^ Problem_1 :Name SimpleAddition :Memo nil :Domain Arithmetic
  :Start nil :Until nil :State Gap_1 :Status open :Result nil]
```

Problem States

A problem state, or problem context defines the initial situation for the problem.

```
(relation State) ; Marker interface
```

In problem solving, there are several conventional states: gaps, lacunae, obstacles, novelties, and processes.

Gaps

A **gap** is a spatio-temporal distance between an initial state of facts and a desired state of goals. A **plan** or set of actions is needed to specify the solution that bridges the gap between the states

```
(relation Gap uses State :Facts :Goals )

(assume Gap
  :Facts  premise ... ; initial facts
  :Goals  premise ... ; desired goals
)
```

Lacunas

A **lacuna** is a logical distance between a set of premises and a set of conclusions (called theorems). A **proof** or **argument** is needed to fill the lacuna.

```
(relation Lacuna uses State :Premises :Theorems )

(assume Lacuna
  :Premises  premise ... ; initial statements
  :Theorems  premise ... ; desired statements
)
```

Obstacles

An **obstacle** is a prediction failure between an expected state of affairs and an actual state of affairs. An **explanation** is required which identifies the obstacle and its cause.

```
(relation Obstacle uses State :Expected :Observed)

(assume Obstacle
  :Expected  premise ... ; expected outcome
  :Observed  premise ... ; actual outcome
)
```

Novelties

A **novelty** is a poorly understood condition which needs to be **explored** through conjecture, experimentation, and variation. Building a theory containing actions and inferences about the novelty bridges the gap in understanding.

```
(relation Novelty uses State :Facts :Found )

(assume Novelty
  :Facts  premise ... ; initial facts
  :Found  premise ... ; discoveries
)
```

Processes

A **process** is a set of facts and phases through which the facts must undergo.

```
(relation Process uses State :Phases :Doing :Facts )

(assume Process
  :Phases phase ...    ; parts of the process
  :Doing  phase        ; the currently executing phase
  :Facts  premise ...  ; initial facts
)
```

Operations on States

To consider an operation on a state is to obtain the successor state after additions and deletions are made to a clone of the state. For each operation a custom binding method and a custom successor method are stored as anonymous functions within the operation itself. These are helper functions.

```
(facility convertVariableToSymbol {literal ?variable}
   ; converting variable $x to symbol ?x
   (reference (symbol (rest ($ ?variable)))))


(facility mapVariablesToSymbols {list ?variables}
  ; converts variables {$x $y $z} to symbols {?x ?y ?z}
  (coalesce ?variable in  ?variables
     (given {literal ?variable}
        (expression ?variable (convertVariableToSymbol ?variable)))))


(nix Operation) ; delete prior definition

(relation Operation
  :Name    :Parameters     :Context     :Effects     !bindings      !successor
  !onNew
    (function Operation_onNew {Operation ?me}
      ;
      ; create a custom bindings anonymous function
      ;
      (let ?mappings (mapVariablesToSymbols (:Parameters ?me))
           ?template (:Context ?me)
           ?context   (replace ?template ?mappings))
      (put ?me
           !bindings
           (` (given {Operation ?operation State ?state}
                (within ?state :Facts ,_ ?context
                   list ,_ (coalesce ?variable ?symbol in ?mappings
                             (expression ?variable (` (eval ,?symbol)))))))))
      ;
      ; create a custom successor anonymous function
      ;
      (put ?me
           !successor
           (` (given {Operation ?operation State ?state list ?bindings}
                (bind ?bindings ,_ (mapVariablesToSymbols
                                     (:Parameters ?operation)))
                (let ?successor (copy ?state :Prior ?state))
                (enq ?successor :Facts each
                     ,_ (map (:Affirm (:Effects ?operation))
                             (given {?addition} (known ?addition))))
                (deq ?successor :Facts each
                     ,_ (map (:Remove (:Effects ?operation))
                             (given {?deletion} (known ?deletion))))
                (return ?successor))))
    )


(function consider {State ?state Operation ?operation}
  (if (thing-p (var ?bindings (!bindings ?operation ?state)))
      (!successor ?operation ?state ?bindings)))
```

```
> (assume Operation
      :Name PickUp
      :Parameters {$ob}
      :Context  { [On :Above CLEAR :Below $ob]
                  [On :Above $ob :Below TABLE]
                  [Has :What ARM :Item EMPTY]}
      :Effects  { :Affirm { [Has :What ARM :Item $ob] }
                  :Remove { [On :Above CLEAR :Below $ob]
                            [On :Above $ob :Below TABLE]
                            [Has :What ARM :Item EMPTY] }} )

.: Operation_1

> (premise Operation_1)

.: [Operation ^ Operation_1
     :Name PickUp
     :Parameters {$ob}
     :Context {On_1 On_2 Has_1 }
     :Effects {:Affirm {Has_2} :Remove {On_1 On_2 Has_1}}
     !bindings
       (given {Operation ?operation State ?state}
         (within ?state :Facts
             [On :Above CLEAR :Below ?ob]
             [On :Above ?ob :Below TABLE]
             [Has :What ARM :Item EMPTY]
          list $ob ?ob))
     !successor
        (given {Operation ?operation State ?state list ?bindings}
          (bind ?bindings $ob ?ob)
          (let ?successor (copy ?state :Prior ?state))
          (enq ?successor :Facts each
            (known [Has :What ARM :Item ?ob]))
          (deq ?successor :Facts each
             (known [On :Above CLEAR :Below ?ob])
             (known [On :Above ?ob :Below TABLE])
             (known [Has :What ARM :Item EMPTY]))
          (return ?successor))
     !onNew  Operation_onNew]
```

Note that the **assume** function finds existing thought identifiers and uses them before creating new thought identifiers. The **!onNew** method is run after all thoughts have been processed.

```
> (assert Operation
      :Name PickUp
      :Parameters {$ob}
      :Context { [On :Above CLEAR :Below $ob]
                 [On :Above $ob :Below TABLE]
                 [Has :What ARM :Item EMPTY] }
      :Effects  { :Affirm { [Has :What ARM :Item $ob] }
                  :Remove { [On :Above CLEAR :Below $ob]
                            [On :Above $ob :Below TABLE]
                            [Has :What ARM :Item EMPTY] }} )

.: Operation_2

> (premise Operation_2)

.: [Operation ^ Operation_2
     :Name PickUp
     :Parameters {$ob}
     :Context {On_3 On_4 Has_3}
     :Effects {:Affirm {Has_4} :Remove {On_5 On_6 Has_5}}
     !bindings
       (given {Operation ?operation State ?state}
         (within ?state :Facts
             [On :Above CLEAR :Below ?ob]
             [On :Above ?ob :Below TABLE]
             [Has :What ARM :Item EMPTY]
          list $ob ?ob))
     !successor
        (given {Operation ?operation State ?state list ?bindings}
          (bind ?bindings $ob ?ob)
          (let ?successor (copy ?state :Prior ?state))
          (enq ?successor :Facts each
            (known [Has :What ARM :Item ?ob]))
          (deq ?successor :Facts each
             (known [On :Above CLEAR :Below ?ob])
             (known [On :Above ?ob :Below TABLE])
             (known [Has :What ARM :Item EMPTY]))
          (return ?successor))
     !onNew  Operation_onNew]
```

Note that the **assert** function creates new thought identifiers regardless of whether they already exist in the knowledge base.

6. Foundation Modules

The SubThought Premise is divided into several modules which contain built-in (i.e., intrinsic) functions. Each module has a particular area of responsibility. For example, the IO module defines file, database, and messaging operations while the knowledge base module (KB) defines storage functions.

Apex

This module contains the global and dynamic environments.

Base

This module provides control structures and intrinsics.

IO

This module contains messaging, file, and the console functions.

KB

This module provides access functions to knowledge bases.

Math

This module provides arithmetic, trigonometric, and statistical functions.

Peer

This module provides inter-agency and syndicate communication.

Tasks

This module has functions that manage asynchronous and concurrent tasks.

User

This module contains the default environment.

7. Quick Reference

A synopsis of each function follows.

1	(-- *number*)	Decrements a number by 1.
2	(- *number* ...)	Subtraction.
3	(@ *container place* ...)	Retrieves elements from a sequence or assortment.
4	(# *container*)	Returns the number of elements.
5	($ *value* ...)	Creates a new string with intervening spaces .
6	($$ *value* ...)	Creates a new string by eliding arguments.
7	(% *command*)	Performs an operating system command
8	(& *value* ...)	Merges arguments into a new sequence.
9	(&& *value* ...)	Merges arguments into a new sequences and removes nils.
10	(* *number number* ...)	Mulitplication.
11	(** *base exponent*)	Exponentiation.
12	(/ *dividend divisor* ...)	Division.
13	(// *number degree*)	*N*th Root.
14	(/~ $value_1$ $value_2$)	Calculates the dissimilarity between two values.
15	(/= $value_1$ $value_2$ ...)	Returns yes if any values are not equal.
16	(~ $value_1$ $value_2$)	Calculates the similarity between two values.
17	(' *variant*)	Quotes a variant.
18	(` *variant*)	Expands a variant by substituting values.
19	(+ *number number* . . .)	Addition.
20	(++ *number*)	Increments a number by 1.
21	(<-- *function symbol value* ...)	The value before tying a symbol to a new value.
22	(< *value value* ...)	Yes if each value is less than the next.
23	(<= *value value* ...)	Yes if each value is less or equal to the next.
24	(<== *container function place value* ...)	The value before replacement.
25	(= *value value* ...)	Yes if each value is equal to the next.
26	(==> *container function place value* ...)	The value after replacement.
27	(> *value value* ...)	Yes if each value is greater than the next.
28	(--> *function symbol value* ...)	The value after tying a symbol to a new value.
29	(>= *value value* ...)	Yes if each value is greater or equal to the next.
30	(\n *quantity*)	Returns a string containing one or more new lines.
31	(\s *quantity*)	Returns a string containing one or more spaces.
32	(^ *thought*)	Returns a premise identifier.
33	(abolish *variables environment*)	Deletes variables from an environment.
34	(abort *tasks*)	Forcibly terminates one or more tasks
35	(about)	Provides system and version information.
36	(abs *number*)	Calculates the absolute value.
37	(absent *premise* ...)	True if the premises are absent in the knowledge base.
38	(acosecant number geometry metrum)	Calculates the inverse cosecant.
39	(acosine number geometry metrum)	Calculates the inverse cosine.
40	(acotangent number geometry metrum)	Calculates the inverse cotangent.
41	(actual *symbol*)	Returns the underlying symbol for a reference.
42	(add *assortment entry* ...)	Modifies an assortment by adding entries.
43	(address *url*)	Returns the address of a URL web resource.
44	(agent *name job url handler delay*)	Creates an agent.
45	(align *sequence ordering*)	Returns a sorted list using the provided ordering.
46	(alike *value value* ...)	Yes if each value has the same taxon.
47	(all *condition* ...)	Yes if all relevant knowledge satisfies all of the conditions.
48	(alphabetic-p *value*)	Yes if the first position of a string is alphabetic.

49	(**alphanumeric-p** *value*)	Yes if the first position of a string is whitespace.
50	(**and** *Boolean Boolean* **...**)	Logical conjunction.
51	(**any** *condition* **...**)	Yes if some relevant knowledge satisfies all of the conditions.
52	(**append** *sequence value* **...**)	Inserts values at the end of a sequence.
53	(**apply** *function arguments environment*)	Applies a function to a list of arguments.
54	(**arity** *function kind*)	Returns the number of variables in a function's parameter list.
55	(**array** *dimensions option*)	Returns a multi dimensional list.
56	(**asecant** number geometry metrum)	Calculates the inverse secant.
57	(**asine** number geometry metrum)	Calculates the inverse sine.
58	(**ask** *who message timeout default*)	Sends a message to a recipient and awaits a response.
59	(**assert** *relation description* **...**)	Creates a thought containing nested premises using new.
60	(**assume** *relation description* **...**)	Creates a thought containing nested premises using knew.
61	(**at** *reference*)	Returns the variant value located at the reference on the heap..
62	(**atangent** *number geometry metrum*)	Calculates the inverse tangent.
63	(**attach** *knowledge*)	Registers a knowledge base.
64	(**average** *symbol* **...** *binding sequence expression* **...**)	The arithmetic mean of a sequence.
65	(**avg** *value* **...**)	The arithmetic mean.
66	(**await** *task timeout default*)	Returns the result of a task.
67	(**before-p** *interval$_1$ interval$_2$ tolerance*)	Yes if an interval finishes before a second interval.
68	(**beginning** *interval*)	Returns the beginning instant of an interval
69	(**bespoke** *name coined* **...**)	Defines a bespoke group of coined literals.
70	(**best** *probe candidates option* **...**)	Returns the best matching candidates.
71	(**beyond** *sequence position*)	Yes if a position is outside a sequence.
72	(**big** *value*)	Converts a value to a big number.
73	(**bind** *container assignment* **...**)	Assigns variables to values in sequences or assortments.
74	(**bindings** *pattern*)	Returns a list of symbol value pairs for an environment or premise.
75	(**bitwise** *number op* **...**)	Performs bitwise operations.
76	(**bound** *function*)	Returns the variables that are bound in a function.
77	(**bound-p** *symbol*)	Yes if a symbol has a value.
78	(**bracket** *value*)	Converts a Boolean or truth value to an integer.
79	(**break** *value*)	Terminates a loop.
80	(**busy-p** *task*)	Yes if a task has not completed.
81	(**but** *sequence quantity*)	Creates a subsequence of all except the last elements.
82	(**bye**)	Terminates the interpreter.
83	(**call** *value* **...**)	Creates an unevaluated call.
84	(**can** *expression result error*)	Evaluates an expression while suppressing failures.
85	(**cancel** *tasks option* **...**)	Cooperatively cancels tasks.
86	(**cancelled-p** *task*)	Yes if a task has been cancelled.
87	(**canonify** *variant*)	Makes equal values identical.
88	(**capitalize** *string option*)	Capitalizes a string.
89	(**case** *probe clause* **...** *else-clause*)	Branches execution based on a value.
90	(**categorize** *sequence predicate* **...**)	Creates a list of equivalence sets.
91	(**cede** *reference*)	Transfers a value between variables.
92	(**ceiling** *number*)	Rounds a number towards positive infinity.
93	(**certify** *module suite ...*)	Evaluates loaded test suites returning QED or a list of issues.
93	(**char** *unicode*)	Converts a Unicode value into a one position string.
94	(**choose** *sequence selector transform*)	Returns the arguments satisfying a function.
95	(**clear** *container*)	Eliminates all entries from an assortment or sequence.
96	(**clip** *value minimum maximum*)	Returns a value within a clipped range.
97	(**clone** *atom modification* **...**)	Clones an atom with possible modifications.
98	(**close** *url*)	Closes a file or data url.
99	(**closure** *scope type name params expression* **...**)	Creates a function or macro with a defined environment.
100	(**coalesce** *symbol* **...** *gate expression* **...**)	Returns a merged sequence.
101	(**collect** *symbol* **...** *gate expression* **...**)	Returns a transformed sequence.

102	(**collection** *element* **...**)	Returns an unorderd collection of elements.
103	(**combine** *assortment entry* **...**)	Creates a new assortment by combining entries.
104	(**common** *sequences*)	Creates a sequence of common elements among subsequences.
105	(**comparable-p** *value1 value2*)	Returns yes if the values can be compared.
106	(**compare** *value1 value2*)	Returns < (less), = (equal), or > (greater).
107	(**compile** *artifact*)	Compiles an expression.
108	(**complete** *expression* **...**)	Runs expressions concurrently until completion.
109	(**complex** *real imaginary*)	Converts a value to a complex number.
110	(**compose** *functions arguments*)	Applies functions in reverse order using the arguments.
111	(**conceive** *knowledge*)	Creates a knowledge base.
112	(**concurrent** *expression* **...**)	Evaluates expressions asynchronously.
113	(**configuration** *url association* **...**)	Creates a configuration file.
114	(**confirm** *condition* **since** *reason*)	Tests that a condition is yes.
115	(**confute** *condition* **since** *reason*)	Tests that a condition is no.
116	(**constant** *assignment* **...**)	Creates a constant.
117	(**contains** *assortment value*)	Yes if a value is present in an assortment.
118	(**contents** *result*)	Creates an expression from a sequence or assortment.
119	(**continue** *result*)	Continues to the next iteration of a loop.
120	(**convertible-p** *value taxon*)	Yes if the value can be converted to the taxon.
121	(**copy** *sequence count*)	Copies a sequence.
122	(**copyright**)	Displays copyright information.
123	(**correlate** *list1 list2*)	Finds the correlation coefficient of two lists.
124	(**cosecant** number geometry metrum)	Calculates the cosecant.
125	(**cosine** number geometry metrum)	Calculates the cosine.
126	(**cotangent** number geometry metrum)	Calculates the cotangent.
127	(**count** *symbol* **...** *binding sequence* **as** *counter expression* **...**)	Counts the iterations and returns the last expression.
128	(**critical** *locks option* **...** *expression* **...**)	Serializes evaluations across regions of code.
129	(**current** *series*)	The current element of a series.
130	(**cut** *assortment key* **...**)	Removes elements from assortments by key.
131	(**data** *option* **...**)	Creates a data url.
132	(**date** *yr mth day hrs mins secs zone zmins*)	Creates a new date or returns the current date.
133	(**declared-p** *symbol environment*)	Yes if a symbol exists in an environment.
134	(**decode** *source format*)	Decodes a string.
135	(**default** *value* **...**)	Returns the first non-nil value.
136	(**definitions** *environment*)	Retrieves literal value pairs in an environment.
137	(**defunct** *function*)	Undefines a function in a module.
138	(**degrees** *radians*)	Converts radians into degrees.
139	(**delete** *sequence value* **...**)	Modifies sequence by deleting values.
140	(**dependencies** *module*)	Returns a module's dependencies.
141	(**deq** *container place option*)	Removes an element from an embedded sequence.
142	(**describe** *literal*)	Returns descriptions of a literal.
143	(**detach** *knowledge*)	Unregisters a knowledge base.
144	(**difference** *sequence1 sequence2*)	Returns the set difference of two sequences.
145	(**differences** *sequence1 sequence2*)	Returns the symmetric difference of two sequences.
146	(**did** *expression*)	Evaluates an expression and surpresses failures.
147	(**digit** *number position*)	Returns the digit in the specified position of a number.
148	(**digit-p** *value*)	Yes if the first position of a string is a digit.
149	(**digits** *number*)	Returns the digits comprising a number.
150	(**dimensions** *sequence*)	Returns the lengths of each dimension in a sequence.
151	(**discard** *module*)	Discards a module.
152	(**disjoint-p** *sequence sequence* **...**)	Yes if no elements are common among sequences.
153	(**distinct** *sequence*)	Creates a new sequence without duplicate elements.
154	(**dstribute** *sequence function result*)	Applies a function to a sequence in parallel.
155	(**divide** *dividend divisor default*)	Performs safe division.
156	(**divisible-p** *dividend divisor*)	Yes if the divisor evenly divides the dividend.

157	(**do** *expression* **…** **resume** *tag resumption* **…** **finally** *cleanup* **…**)	Evaluates expressions and handles escapes.
158	(**domain** *name element* **…**)	Creates a package containing relations and rules.
159	(**domains**)	Returns a list of defined domains.
160	(**done**)	Terminates a generator.
161	(**drop** *index*)	Removes a relation index.
162	(**dump** *knowledge option* **…**)	Writes knowledge expressions to a file.
163	(**duplicate** *source target option*)	Duplicates a file or contents of a folder.
164	(**during-p** $interval_1$ $interval_2$ *tolerance*)	Yes if an interval occurs within a second interval.
165	(**dynamic** *assignments expression* **…**)	Creates a dynamic environment for variables.
166	(**e**)	Returns Euler's number, 2.718281828459045.
167	(**each** *symbol* **…** *binding reversal sequence* **…** *premises option* **…** *action*)	Combines for and with to iterate over sequences to match patterns against the knowledge base.
168	(**encode** *source format*)	Encodes a string.
169	(**enq** *container place option*)	Inserts an element into an embedded sequence.
170	(**ensure** c*heck* **…**)	Performs type checking.
171	(**entries** *assortment*)	Retrieves key value pairs for an assortment.
172	(**enum** *enumeration key*)	Retrieves an enumeration value.
173	(**enumeration** *name element* **…**)	Defines an enumerated assortment.
174	(**enumerations**)	Returns a list of all enumerations.
175	(**environment** *parent entry* **…**)	Creates a new context for variables and functions.
176	(**epoch** *time*)	Returns a Unix epoch or the current epoch.
177	(**eradicate** *knowledge*)	Deletes a knowledge base.
178	(**erase** *file option* **…**)	Deletes files or folders.
179	(**escape** *tag*)	Escapes a do intrinsic
180	(**eval** *expression environment*)	Evaluates an expression.
181	(**even-p** *number*)	Yes if the number is even.
182	(**every** *symbol* **…** *binding sequence expression … test*)	Yes if a predicate is yes for every element in a sequence.
183	(**exactly** *quantity* **of** *sequence clause* **within** *margin*)	Yes if a number of elements satisfy a clause.
184	(**exchange** *sequence substitution* **…**)	Creates a new sequence with values swapped.
185	(**excludes** *sequence element option* **…**)	Yes if a sequence does not contain an element.
186	(**exists** *premise* **…**)	True if the premises exist in the knowledge base.
187	(**exit** *value*)	Explicitly ends a task using a return value.
188	(**exponential** *number significand*)	Calculates the base ten exponent of a number.
189	(**expression** *value* **…**)	Creates an expression.
190	(**extend** *module expression* **…**)	Adds new definitions to a module.
191	(**external** *scope name params doc option* **…**)	Declares a function implemented in a native external library.
192	(**facility** *name parameters expression* **…**)	Creates a private function in a module.
193	(**few** *symbol* **…** *binding sequence expression … test*)	Yes if a predicate is yes for less than half the elements in a sequence.
194	(**file** *option* **…**)	Opens or creates a file.
195	(**files** *option* **…**)	Returns a list of file names.
196	(**fill** *symbol* **from** *start* **to** *end* **by** *increment* **with** *expression*)	Fills a list with the result of an expression.
197	(**filter** *sequence predicate*)	Returns a sequence of elements that satisfy a predicate.
198	(**find** *features candidates option* **…**)	Returns the best matching candidates.
199	(**finishes-p** $interval_1$ $interval_2$ *tolerance*)	Yes if two intervals finish together.
200	(**finishing** *interval*)	Returns the finishing instant of an interval.
201	(**fix** *declaration* **…**)	Adds variables to the current environment returns yes.
202	(**float** *value*)	Converts a value to a floating number.
203	(**floor** *number*)	Rounds a number towards negative infinity.
204	(**fold** *symbol* **…** **in** *sequence* **into** *expression* **…**)	Tranforms a sequence into a value.
205	(**folder** *option* **…**)	Creates or locates a folder in the file system.
206	(**folders** *option* **…**)	Returns a list of sub folders.
207	(**for** *symbol* **…** *binding reversal sequence* **…** *expression* **…**)	Iterates over the elements in a sequence.

208	(**forever** *expression* **…**)	Repeatedly evaluates expressions.
209	(**forgo** *dependency module*)	Removes a dependent module.
210	(**format** *template value* **…**)	Formats a string.
211	(**fractional** *number*)	Returns the fractional portion of a number.
212	(**free** *resources*)	Releases resources.
213	(**from** *criteria* **…** *options* **…** *action*)	Creates bindings using the knowledge base.
214	(**full** *first second*)	Yes if values are equal or either value is nil.
215	(**full-p** *sequence*)	Yes if a container has the maximum number of elements.
216	(**function** *scope name params returning expression* **…**)	Creates a public function in a module.
217	(**functions** *module*)	Returns a list of functions defined in a module.
219	(**gather** *symbol* **…** *binding sequence expression … test*)	Returns a sequence of elements that satisfy a predicate.
220	(**generator** *name parameters expression* **…**)	Creates a series generator.
221	(**get** *container key* **…**)	Retrieves a value using one or more keys.
223	(**given** { *parameter* **…** **}** *expression* **…**)	Creates an anonymous function.
224	(**global** *assignment* **…**)	Creates global variables.
225	(**go** *label*)	Transfers control to a label within a function.
226	(**gone** *url*)	Yes if a file, folder, or ur does not exist.
227	(**grimoire** *name version lexicals tokemes tree packages start taxons rendering renderings*)	Creates an interpretable language grammar.
228	(**grok** *url option*)	Reads expressions from a url or file.
229	(**group** *list* **by** *key* **…** **as** *symbol expression* **…**)	Combines sublists by position or key.
230	(**has** *assortment key*)	Yes if a key is present in an assortment.
231	(**hash** *value option* **…**)	Computes a hash code.
232	(**help** *function*)	Describes a function.
233	(**here** *label*)	Creates a local label within a function.
234	(**hyperlink** *option* **…**)	Creates a hyperlink.
235	(**id** *resource*)	Returns a resource number.
236	(**identical-p** *value value* **…**)	Yes if all values occupy the same memory location.
237	(**identity** *value*)	Returns the value itself.
238	(**idiom** *pair* **…**)	Creates a list of key value pairs.
239	(**idle** *duration*)	Pauses for a specified interval.
240	(**if** *condition expression … or-clause … else-clause*)	Branched conditional evaluation.
241	(**imaginary** *value*)	Converts a value to an imaginary number.
242	(**in** *value container option* **…**)	Yes if a value is in a sequence or assortment.
243	(**includes** *sequence element option* **…**)	Yes if a value is in a sequence.
244	(**index** *relation slot* **…**)	Creates an index on slots of a relation.
245	(**indices** *relation*)	Returns a list of indices for a relation.
246	(**infer** *domain*)	Applies rules in all domains or a specific doman.
247	(**infix** *sequence delimeter*)	Creates a new sequence with interposed delimeters.
248	(**inside** *sequence position*)	Yes if a position is inside the bounds of a sequence.
249	(**insert** *sequence position value* **…**)	Creates a new sequence by nserting values.
250	(**insort** *sequence value option* **…**)	Inserts a value into a sorted sequence.
251	(**instantiate** *expression*)	Creates an expression with premises in place of thoughts.
252	(**integer** *value*)	Converts a value to an integer.
253	(**interior** *value*)	Creates an expression from a sequence or assortment.
254	(**interleave** *sequence sequence* **…**)	Merges arguments into a new sequence.
255	(**intersection** *sequence sequence* **…**)	Creates a sequence of common elements.
256	(**intersects-p** *sequence sequence* **…**)	Yes if any elements are common among sequences.
257	(**interval** *start finish*)	Creates a time interval.
258	(**into** *relation thoughts*)	Creates new thoughts based on existing thoughts.
259	(**invoke** *call environment*)	Invokes a call.
260	(**is** *value predicate*)	Yes if the value is yes or if the applied predicate returns yes.
261	(**json** *value*)	Converts a value to a Javascript Object Notation string.

262	(**junction** *scope name params expression* **…**)	Creates a public function where arguments are evaluaed in parallel.
263	(**key** *assortment value*)	Finds the key for a value in an assortment.
264	(**keys** *assortment*)	Creates a list of keys for an assortment.
265	(**keywords** *name reserved …*)	Creates a grouping of reserved keywords..
266	(**kinds** *relation*)	Returns the subtypes of a relation..
267	(**knew** *relation criterion* **…**)	Finds or creates a thought.
268	(**knowledge** *option* **…**)	Finds or creates a knowledge base.
269	(**known** *premise* **…**)	Finds or creates thoughts using premises.
270	(**lacks** *assortment key*)	Yes if a key is absent from an assortment.
271	(**last** *sequence count*)	Creates a subsequence of the last elements.
272	(**left** *first second*)	Yes if values are equal or the second value is nil.
273	(**let** *assignment* **…**)	Adds variables to the current environment returning yes.
274	(**lexemes** *string*)	Creates an uppercase string with spaces between words.
275	(**lexicon** *association* **…**)	Creates a lexicon.
276	(**lexicons**)	Creates a list of all lexicons.
277	(**license**)	Prints the software license.
278	(**like** *sequence pattern*)	Compares a pattern to a sequence.
279	(**list** *value* **…**)	Creates a list.
280	(**literal** *value*)	Creates a literal.
281	(**local** *assignments expression* **…**)	Creates a task level scope.
282	(**location** *symbol*)	Sets a symbol to the current location.
283	(**log** *number base*)	Logarithm.
284	(**long** *value*)	Converts a value to a long number.
285	(**loop** *expression* **…** *gate*)	Repeatedly evaluates expressions.
286	(**lowercase** *string*)	Converts a string to lower case.
287	(**macro** *name parameters expression* **…**)	Creates a public macro in a module.
288	(**macros** *module*)	Creates a list of macros defined in a module.
289	(**make** *prototype term* **…**)	Creates a record by creating a relationship if it does not exist.
290	(**map** *sequence* **…** *function*)	Applies a function to elements across sequences.
291	(**mask** *number operation operand*)	Combines the bits of two numbers.
292	(**match** *problem*)	Matches rules in all open problems or a specific problem.
293	(**max** *value* **…**)	Finds the maximum element.
294	(**maximum** *symbol* **…** *binding seq expr* **…**)	Finds the maximum element.
295	(**may** *expression*)	Evaluates an expression while surpressing failures.
296	(**me** *value*)	Returns the heap reference to the value..
297	(**median** *value* **…**)	Finds the middle value of a sequence.
298	(**meets-p** $interval_1$ $interval_2$ *tolerance*)	Yes if an interval finishes when a second interval starts.
299	(**meron** *value*)	Returns the meronomic prototype of a value.
300	(**meron-p** *value meron*)	Yes if the value is a meron or is of the specific meronomic type.
301	(**meronomy** *value*)	Returns the meronomic prototypes of a value.
302	(**method** *scope name params expression* **…**)	Creates a public method in a prototype.
303	(**mezo** *value*)	Converts a value to 32 bit unsigned integer.
304	(**mid** *sequence start stop skip*)	Copies a subsequence of a sequence.
305	(**min** *value* **…**)	Finds the minimum element.
306	(**minimum** *symbol* **…** *binding seq expr* **…**)	Finds the minimum element.
307	(**missing** *assortment value*)	Yes if a value is absent from an assortment.
308	(**mod** *dividend modulus*)	Finds the remainder after division.
309	(**modular** *module expression* **…**)	Evaluates expressions in module's scope.
310	(**module** *name definition* **…**)	Creates a module.
311	(**modules**)	Creates a list of known modules.
312	(**moi**)	Returns the currently executing machine resource.
313	(**moment** *units secs mins hours days year*)	Creates a moment or returns the current moment.
314	(**more-p** *sequence*)	Yes if a container has more than zero elements.
315	(**morph** *arguments functions*)	Applies functions in succession using the arguments.

316	(**most** *symbol* **…** *binding sequence expression … test*)	Yes if the predicate is yes for more than half of the elements in a sequence.
317	(**move** *source destination*)	Moves or renames one or more files.
318	(**my**)	Returns the current cell resource.
319	(**nall** *condition* **…**)	Yes if not all relevant knowledge satisfies all of the conditions.
320	(**nand** *Boolean Boolean* **…**)	Negated logical conjunction.
321	(**negative-p** *number*)	Yes if a number is negative.
322	(**nevery** *symbol* **…** *binding sequence expression … test*)	Yes if the predicate is no for any element in the sequence.
323	(**new** *prototype term* **…**)	Creates a record.
324	(**next** *series*)	Advances a series to the next element.
325	(**ngrams** *string length*)	Creates a list of substrings.
326	(**nil-p** *value*)	Yes if a value is nil.
327	(**nix** *prototype*)	Deletes a prototype.
328	(**none** *symbol* **…** *binding sequence expression … test*)	Yes if the predicate is no for all sequence elements.
329	(**nor** *Boolean Boolean* **…**)	Negated logical disjunction.
330	(**not** *value predicate*)	Yes if a value is no, or if the applied predicate returns no.
331	(**notany** *condition* **…**)	Yes if no relevant knowledge satisfies all of the conditions.
332	(**nothing**)	Returns nothing.
333	(**noumenon** *value*)	Returns the raw representation of a value.
334	(**null-p** *value*)	Yes if a value is nothing, nil, or void.
335	(**odd-p** *number*)	Yes if a number is odd.
336	(**old** *thought*)	Deletes a thought.
337	(**omit** *sequence value* **…**)	Creates a new sequence with values removed.
338	(**on** *condition yes-case no-case*)	Branched conditional evaluation.
339	(**only** *variables expression* **…**)	Creates a restricted scope.
340	(**open** *url*)	Opens a file or data url.
341	(**or** *Boolean Boolean* **…**)	Logical disjunction.
342	(**out** *value sequence option* **…**)	Yes if a value is absent from a sequence.
343	(**overlaps-p** $interval_1$ $interval_2$ *tolerance*)	Yes if an interval finishes after a second interval starts.
344	(**pad** *sequence length value side*)	Creates a new padded sequence.
345	(**parse** *readable option* **…**)	Creates an expression from a readable sequence.
346	(**parseable** *readable desired*)	Calculates the number of expressions that can be read.
347	(**parsed-p** *readable option* **…**)	Creates an expression from a readable sequence.
348	(**partition** *sequence pivot comparer*)	Creates a list of equivalence sets for a pivot element.
349	(**parts** *relation*)	Returns the components of the relation.
350	(**path** *folder separator file*)	Concatenates a folder and file name.
351	(**pattern** *expression* **…**)	Creates a pattern containing the supplied expressions.
352	(**peek** *sequence priority*)	Returns the next element in a sequence.
353	(**peep** *task option duration*)	Retrieves a message from a task mailbox.
354	(**perform** *environment method instance* **…**)	Invokes a method within an environment.
355	(**pi**)	Returns 3.141592653589793.
356	(**pick** *sequence quantity*)	Creates a list of randomly selected elements.
357	(**pipe** *option* **…**)	Creates a pipe.
358	(**pop** *sequence priority*)	Attempts to remove an element from a sequence.
359	(**posit** *premise* **…**)	Creates possibly redundant thoughts using premises.
360	(**position** *sequence element option* **…**)	Finds the position of an element or subsequence in a sequence.
361	(**positions** *sequence*)	Returns positions of an element in a sequence or position value pairs.
362	(**positive-p** *number*)	Yes if a number is greater than zero.
363	(**premise** *value*)	Creates a premise representation of a value.
364	(**probability** *events A operation B*)	Calculates the probability over a series.
365	(**problem** *name* **memo** *description* **domain** *realm* **start** *time* **until** *time* **state** *state* **status** *status* **result** *result*)	Creates a problem to be solved by the rules enginie.

366	(**problems**)	Returns a list of problems.
367	(**procedure** *scope name params expression* **...**)	Creates a public function that returns \%Nothing within a module.
368	(**proceed** *location* **set** *assignment* **...**)	Transfers control to a location.
369	(**product** *symbol* **...** *binding sequence expression* **...**)	Multiplies an expression over a range of values.
370	(**pson** *string*)	Creates a Premise Structured ObjectNotation string.
371	(**punctuation-p** *string*)	Yes if the first position of a string is punctuation.
372	(**puny** *value*)	Converts a value to a 16 bit signed integer.
373	(**push** *sequence value position*)	Modifies a sequence by inserting a value.
374	(**put** *container entry* **...**)	Updates values in an assortment or sequence.
375	(**qualified** *module function*)	Prepends a module to a function name.
376	(**qualifiers** *identifier*)	Creates a list of modules associated with an identifier.
377	(**quantify** *sequence predicate option* **...**)	Returns a quantifier for elements satisfying a test.
378	(**quantity** *symbol* **...** *binding sequence expression ... test*)	Returns the number of elements satisfying a test.
379	(**queue** *value* **...**)	Creates a queue.
380	(**radians** *degrees*)	Converts degrees to radians.
381	(**radix** *number base*)	Converts a number to a base.
382	(**random** *lower* **to** *upper precision*)	Creates a random number.
383	(**range** *first* **to** *last* **by** *increment*)	Creates a list of numbers.
384	(**rational** *numerator denominator*)	Converts values to a rational number.
385	(**read** *file count bits*)	Creates a string or stream.
386	(**ready-p** *task*)	Yes if a task has completed.
387	(**real** *value*)	Converts a value to a real number.
388	(**reclaim**)	Performs garbage collection.
389	(**reduce** *sequence function*)	Converts a sequence into a value.
390	(**reference** *symbol*)	Returns a reference to a symbol.
391	(**relation** *name inclusion composition inheritance membership definition* **...**)	Creates a relation.
392	(**relations**)	Returns a list of defined relations.
393	(**release** *locks*)	Releases locks on critical sections.
394	(**remove** *assortment value* **...**)	Creates a new assortment by removing values.
395	(**repeat** *count expression* **...**)	Loops a specified number of times.
396	(**replace** *container entries option*)	Modifies an assortment or sequence by with replacement values.
397	(**require** *module options* **...**)	Retrieves an absent module or domain from a url.
398	(**rescind** *expression* **...**)	Deletes rules.
399	(**reserved** *name item* **...**)	Creates a reserved grouping.
400	(**reset** *series*)	Resets a series for reuse.
401	(**resize** *sequence length default*)	Modifies the length of a sequence.
402	(**rest** *sequence skip*)	Creates a subsequence of all except the first elements.
403	(**retract** *expression* **...**)	Deletes thoughts individually or using a pattern.
404	(**return** *value* **from** *function*)	Terminates a function call.
405	(**reverse** *sequence*)	Creates a reversed sequence.
406	(**right** *first second*)	Yes if values are equal or the first value is nil.
407	(**ring** *option* **...**)	Creates a ring sequence.
408	(**rip** *assortment value* **...**)	Removes elements from assortments by value.
409	(**round** *number*)	Rounds a number to the nearest integer.
410	(**rule** *name* **memo** *description* **domain** *ruleset* **salience** *priority* **problem as** *symbol* **state as** *symbol match expression* **...** *options* **do** *expression* **...** **then** *ruleset*)	Creates a rule.
411	(**rules** *domain*)	Creates a list of all rules in a domain.
412	(**same** *value value* **...**)	Yes if all values are equal or contain equal elements.
413	(**save** *module option* **...**)	Writes module expressions to a file.
414	(**scale** *list scalar* **...** *function*)	Applies a function to a list of numbers and scalar values.

415	(**scatter** *arguments functions*)	Applies functions in parallel returning a list of results.
416	(**scope** *keyval* **…**)	Finds an environment or gets the current environment.
417	(**seal** *prototype*)	Prohibits new records.
418	(**secant** number geometry metrum)	Calculates the secant of the number.
419	(**seconds** *seconds*)	Creates a seconds value or returns seconds since year zero.
420	(**seek** *file position*)	Repositions a file resource to a new read/write location.
421	(**self**)	Returns the currently executing task resource.
422	(**send** *task message option* **…**)	Delivers a message to a task mailbox.
423	(**separator** *kind*)	Creates a separator string.
424	(**series** *iterable*)	Creates a series.
425	(**service** *name url handler*)	Creates and starts a web service.
426	(**set** *manner assignment* **…**)	Assigns variables to values in tandem returning yes.
427	(**settings** *url association* **…**)	Creates a configuration file..
428	(**sever** *assortment key* **…**)	Creates a new assortment by removing keys.
429	(**shift** *number option*)	Moves bits of a number left or right.
430	(**short** *value*)	Converts a value to a 32 bit signed integer.
431	(**shuffle** *sequence seed*)	Creates a reordered sequence.
432	(**sigma** *list*)	Calculates the sigma of a sequence.
433	(**signal** *category attribute* **…**)	Signals a condition.
434	(**significand** *number kind*)	Calculates the modified significand as zero, or a number from 1 to 10.
435	(**signum** *number option* **…**)	Returns the sign of a number as a number, sigil, or word.
436	(**sine** number geometry metrum)	Calculates the sine of a number.
437	(**so** *bindings expression* **…**)	Creates a scope with bindings.
438	(**socket** *option* **…**)	Creates a TCP socket.
439	(**some** *symbol* **…** *binding sequence expression … test*)	Yes if the test is yes for any element in the sequence.
440	(**sort** *sequence ordering option* **…**)	Creates a sorted sequence.
441	(**split** *sequence delimiter*)	Creates a list of subsequences divided at a delimiter.
442	(**stack** *value* **…**)	Creates a stack.
443	(**start** *compute argument* **…**)	Starts an agent, service, or task.
444	(**starts-p** $interval_1$ $interval_2$ *tolerance*)	Yes if two intervals start together.
445	(**statistics** *numbers*)	Finds the mean, median, sigma, and count of a list of numbers.
446	(**step** *symbol* **from** *i limit j* **by** *k expression* **…**)	Loops a symbol over a numeric range to evaluate expressions.
447	(**stream** *streamable*)	Creates a stream.
448	(**string** *value* **…**)	Converts a value to a string.
449	(**structure** *name inclusion* **…** *definition* **…**)	Creates a structure.
450	(**structures**)	Creates a list of all structures.
451	(**sub** *sequence entry* **…**)	Replaces a subsequence within a sequence.
452	(**subset-p** *subset superset*)	Yes if all elements in a subset are in a superset.
453	(**substitute** *sequence entries*)	Modifies a sequence replacing subsequences.
455	(**subsumes-p** *superset subset* **…**)	Yes if all elements in each subset are in the superset.
454	(**suites** *module suite …*)	Loads test suites into a module.
455	(**sum** *value* **…**)	Calculates the arithmetic sum.
456	(**summation** *symbol* **…** *binding seq expr* **…**)	Adds an expression over a set of values.
457	(**supply** *arguments function environment*)	Applies a function to an argument list.
458	(**suppose** *knowledge* **facts** *facts* **action** *goals* **by** *strategies* **via** *operators* **limit** *quantity* **gate** *condition* **within** *timeframe* **before** *timepoint* **options** *options*)	Performs hypothetico-deductive reasoning from facts to goals.
459	(**survey** *format relation slot*)	Creates a list of referents or referent-count pairs.
460	(**swap** *sequence substitution* **…**)	Modifies a sequence by interchanging values.
461	(**symbol** *name*)	Creates a symbol.
462	(**symbols** *environment ancestors*)	Lists the symbols in an environment.
463	(**system** *topic option setting*)	Returrns or sets a system configuration setting.
464	(**take** *sequence position*)	Returns the element removed from a sequence.

465	(**tally** *pattern*)	Returns the number of matches in the knowledge base.
466	(**tangent** number geometry metrum)	Calculates the tangent.
469	(**tarry** *tasks expression* **…**)	Allows tasks to complete on their own.
470	(**task** *scope expression* **…**)	Creates a list of tasks for deferred evaluation.
471	(**tasks**)	Creates a list of all tasks.
472	(**taxon** *value*)	Returns the datatype of a value.
473	(**taxon-p** *value taxon*)	Yes if the value is of the specific taxonomic type.
474	(**taxonomy** *value*)	Returns a list of datatypes for a value.
475	(**teeny** *value*)	Converts a value to an 8 bit unsigned integer.
476	(**tell** *who message timeout*)	Sends a message to a recipient.
477	(**the** *instance slot*)	Retrieves the value of a slot from a tuple or thought.
478	(**there** *url*)	Yes if a file, folder, or url exists.
479	(**thing-p** *value*)	Yes if a value is a thing..
480	(**think** *problem domain*)	Creates a match-infer loop task to solve problems.
481	(**this** *bindings expression* **…**)	Creates a restricted environment using symbol assignments.
483	(**thought** *relation id*)	Finds an extant thought.
484	(**thunk** *expression* **…**)	Creates a thunk in a module.
485	(**thunks** *module*)	Returns a list of thunks defined in a module.
486	(**tick** *nanoseconds*)	Creates a tick or returns nanoseconds since year zero.
487	(**tie** *manner assignment* **…**)	Assigns variables to values returning the last assigned value.
488	(**time** *unit operation time* **…**)	Performs time operations.
489	(**tiny** *value*)	Converts a value to a 16 bit unsigned integer.
490	(**top** *sequence count*)	Creates a subsequence of the first elements.
491	(**transfer** *source destination option* **…**)	Transfers a value from one sequence to another.
492	(**traverse** *symbol binding sequence* **limit** *dimensions expression* **…**)	Loops through the coordinates of a sequence.
493	(**trim** *sequence option* **…**)	Creates a sequence without front and rear delimiters.
494	(**truncate** *number*)	Rounds a number towards zero.
495	(**try** *expression* **…** **learn** *symbol recovery* **… finally** *cleanup* **…**)	Evaluates expressions and handles signals.
496	(**tuple** *value* **…**)	Creates a tuple.
497	(**types** *relation*)	Returns the supertypes of a relation.
498	(**uuid**)	Creates a unique identifier.
499	(**unbound** *function*)	Creates a list of free variables for a function.
500	(**unbound-p** *symbol*)	Yes if a symbol is unbound.
501	(**unicode** *string*)	The Unicode of a string's first position.
502	(**union** *sequence sequence* **…**)	Concatenates sequences removing duplicates.
503	(**union** *sequence sequence* **…**)	Concatenates sequences removing duplicates.
504	(**unique** *prefix*)	Creates a unique literal based on a prefix.
505	(**unless** *condition expression* **…**)	Conditional execution.
506	(**unlike** *sequence pattern*)	Yes if a pattern does not match a sequence.
507	(**unqualified** *function*)	The function literal without the module prefix.
508	(**unseal** *prototype*)	Permits new records for a protoytpe.
509	(**unsigned** *value*)	Converts a value to an unsigned number.
510	(**until** *condition expression* **…**)	Loops expressions until a condition is yes.
511	(**unwrap** *module*)	Permits modifications to a module.
512	(**uppercase** *string*)	Converts a string to uppercase.
513	(**uppercase-p** *value*)	Yes if a string's first position is upper case.
514	(**url** *option* **…**)	Creates a URL resource.
515	(**use** *knowledge expression* **…**)	Evaluates expressions in a knowledge base scope.
516	(**using** *scope expression* **…**)	Accesses a scope.
517	(**values** *assortment*)	Creates a list of values for an assortment.
518	(**var** *assignment* **…**)	Adds variables to the current environment returning last value.
519	(**vector** *atom* **…**)	Creates a vector of atoms.
520	(**version**)	Provides version information.
521	(**void-p** *sequence*)	Yes if a container has zero elements.

522	(**vouch** *task receipt*)	Deletes a receipt uuid from a task's shoebox.
523	(**vouchers** *task*)	Returns a list of unclaimed read receipts in a task's shoebox.
524	(**wait** *duration*)	Pauses for a specified duration.
525	(**wee** *value*)	Converts a value to an 8 bit signed integer.
526	(**when** *condition expression* **…**)	Conditional execution.
527	(**which** *pattern options* **…**)	Finds matches against the knowledge base.
528	(**while** *condition expression* **…**)	Loops expressions while a condition is yes.
529	(**whitespace-p** *value*)	Yes if the first position of a string is whitespace.
530	(**wholes** *relation*)	Returns the containers of a relation.
531	(**with** *premises option* **…** *action expression*)	Processes matches against the knowledge base.
532	(**within** *which slot premises option* **…** *action*)	Yes if a position is inside a sequence.
533	(**wrap** *module*)	Prohibits modifications to a module.
534	(**write** *writeable value* **…**)	Writes values to a writeable resource.
535	(**xnor** *Boolean Boolean*)	Logical equivalance.
536	(**xor** *Boolean Boolean*)	Logical exclusive disjunction.
537	(**yield** *value*)	Returns a result from a series generator.
538	(**zap** *container place* **…**)	Updates associations and sequences with nil values.
539	(**zero-p** *number*)	Yes if a number is zero.
524	(**wee** *value*)	Converts a value to an 8 bit signed integer.
525	(**when** *condition expression* **…**)	Conditional execution.
526	(**which** *pattern options* **…**)	Finds matches against the knowledge base.
527	(**while** *condition expression* **…**)	Loops expressions while a condition is yes.
528	(**whitespace-p** *value*)	Yes if the first position of a string is whitespace.
529	(**wholes** *relation*)	Returns the containers of a relation.
530	(**with** *premises option* **…** *action expression*)	Processes matches against the knowledge base.
531	(**within** *which slot premises option* **…** *action*)	Yes if a position is inside a sequence.
532	(**wrap** *module*)	Prohibits modifications to a module.
533	(**write** *writeable value* **…**)	Writes values to a writeable resource.
534	(**xnor** *Boolean Boolean*)	Logical equivalance.
535	(**xor** *Boolean Boolean*)	Logical exclusive disjunction.
536	(**yield** *value*)	Returns a result from a series generator.
537	(**zap** *container place* **…**)	Updates associations and sequences with nil values.
538	(**zero-p** *number*)	Yes if a number is zero.

Table 1. Function Quick Reference

Appendix A - OPS5 Comparison

The OPS5 programming language was developed by Charles Forgy in the late 1970s. OPS is an acronym for "Official Production System" and it is a rule based programming language primarily used for expert system development. The following is an OPS5 program that implements a model of cars.

OPS5

```
(literalize car
    age           ; new or old
    condition     ; good or junk
)

(p old-not-junk
    (car ^age old ^condition <> good)
--->
    (write (crlf) that is believable))

(p good-not-old
    (car ^age <> old ^condition good)
--->
    (write (crlf) that is possible))

(p new-and-junk
    (car ^age new ^condition junk)
--->
    (write (crlf) There are no new, junk cars))

(copy car ^age new ^condition good)
```

Premise

```
(relation Car
  :Age            ; new or old
  :Condition      ; good or junk
)

(rule  old-not-junk
  with  [Car :Age = old :Condition /= good]
  do    (tell user ($ that is believable \n)))

(rule  good-not-old
  with   [Car :Age /= old :Condition = good]
  do   (tell user ($ that is possible \n)))

(rule  new-and-junk
  with   [Car :Age = new :Condition = junk]
  do   (tell user ($ There are no new, junk cars \n)))

(new Car :Age new :Condition good)
```

Appendix B - CLIPS Comparison

The C Language Integrated Production System, CLIPS, was developed in 1985 by NASA and was based on Charles Forgy's OPS languages and the Automated Reasoning Tool language from Inference Corporation. The example below is derived from https://en.wikipedia.org/wiki/CLIPS.

CLIPS

```
(deftemplate trouble
     (slot name)
     (slot status))


(defrule rule1
     (trouble (name ignition_key) (status on))
     (trouble (name engine) (status wont_start))
     (trouble (name headlights) (status don't_work))
      =>
     (confirm (trouble (name battery) (status faulty))))


(deffacts trouble_shooting
     (trouble (name ignition_key) (status on))
     (trouble (name engine) (status wont_start))
     (trouble (name headlights) (status don't_work)))
```

Premise

```
(relation Trouble
  :Name
  :Status
)

(rule rule1
  with [Trouble :Name = ignitionKey :Status = on]
       [Trouble :Name = engine :Status = WontStart]
       [Trouble :Name = headlights :Status = DontWork]
  do
       (knew Trouble :Name battery :Status faulty)
)

(function troubleshooting
  (new Trouble :Name ignitionKey :Status on)
  (new Trouble :Name engine :Status WontStart)
  (new Trouble :Name headlights :Status DontWork)
)

(troubleshooting)
```

Appendix C - SQL Comparison

Structured Query Language (SQL) is a language for programming relational database management systems (RDMS). SQL was first described by Edgar F. Codd in 1970.

Relationships

In Structured Quey Language, entities and the relationships among them are central to the languge. Entities (and relationships) are defined by **Tables** which have **Columns**.

SQL

```
CREATE TABLE Region (
    RegionID INTEGER NOT NULL,
    Name VARCHAR(50) NOT NULL
);

CREATE TABLE Country (
    CountryID INTEGER NOT NULL,
    Name VARCHAR(50) NOT NULL
);

CREATE TABLE Customer (
    CustomerID INTEGER NOT NULL AUTO_INCREMENT,
    RegionID INTEGER NOT NULL,
    CountryID INTEGER NOT NULL,
);

CREATE TABLE Order (
    OrderID INTEGER NOT NULL AUTO_INCREMENT,
    CustomerID VARCHAR(10)
);

CREATE TABLE Category (
    CategoryID INTEGER NOT NULL AUTO_INCREMENT,
    CategoryName VARCHAR(100) NOT NULL,
    Description MEDIUMTEXT
);

CREATE TABLE Product (
    ProductID INTEGER NOT NULL AUTO_INCREMENT,
    ProductName VARCHAR(100) NOT NULL,
    CategoryID INTEGER,
    UnitsInStock FLOAT,
    UnitPrice FLOAT
    Discontinued  BIT
);

CREATE TABLE Employee (
    EmployeeID INTEGER NOT NULL AUTO_INCREMENT,
         EmployeeName VARCHAR(100) NOT NULL,
    ReportsTo INTEGER,
    Sold FLOAT
);
```

Premise

```
(relation Region
  :Name
)

(relation Country
  :Name
)

(relation Customer
  :Region
  :Country
)

(relation Order
  :CustomerId
)

(relation Category
  :CategoryId
  :CategoryName
  :Description
)

(relation Product
  :ProductId
  :ProductName
  :CategoryId
  :UnitsInStock
  :UnitPrice
  :Discontinued  no
)

(relation Employee
  :EmployeeId
  :EmployeeName
  :ReportsTo
  :Sold
)
```

Select All

A select all query retrieves all attributes of an entity.

SQL

```
SELECT *
  FROM Category
```

Premise

```
(with Category)
```

Selecting a single column

In SQL a single column is selected by specifying the name of the column in a select query.

SQL

```
SELECT CategoryName
  FROM Category
```

Premise

```
(with  [Category as ?c]
 list  (:CategoryName ?c))
```

```
(with  [Category :CategoryName as ?n]
 list  ?n)
```

Selecting multiple columns

In SQL multiple columns are selected by naming each column in a select query.

SQL

```
SELECT CategoryName, Description
  FROM Category
```

Premise

```
(with [Category :CategoryName as ?n :Description as ?d]
 list  {?n ?d})
```

Selecting a calculated column

A calculated column is one in which a function is applied to one or more existing columns in a table.

SQL

```
SELECT LENGTH(CategoryName)
  FROM Category
```

Premise

```
(with [Category  :CategoryName as ?n]
 list (# ?n))
```

Selecting distinct values

SQL

```
SELECT DISTINCT LENGTH(CategoryName)
  FROM Category
```

Premise

```
(distinct
  (with [Category :CategoryName as ?n]
   list (# ?n)))
```

Selecting a scalar value

SQL

```
SELECT MAX(LENGTH(CategoryName))
  FROM Category
```

Premise

```
(with [Category :CategoryName as ?n]
 max  (# ?n)))
```

Filtering results by Equality

SQL

```
SELECT *
  FROM Product
 WHERE UnitsInStock = 0
```

Premise

```
(with [Product :UnitsInStock = 0])
```

Filtering results by Inequality

SQL

```
SELECT *
  FROM Product
 WHERE NOT (UnitPrice > 10)
```

Premise

```
(with [Product :UnitPrice as ?p]
      (not (> ?p 10)))
```

```
(with [Product :UnitPrice <= 10])
```

Filtering results by a range

SQL

```
SELECT *
  FROM Product
 WHERE UnitPrice BETWEEN 5 AND 9
```

Premise

```
(with [Product
       :UnitPrice as ?u
       (<= 5 ?u 9)])
```

Multiple filter conditions

SQL

```
SELECT *
  FROM Product
 WHERE Discontinued = 1
   AND UnitsInStock <> 0
```

Premise

```
(with [Product
       :Discontinued = 1
       :UnitsInStock not 0])
```

Order by value ascending

SQL

```
SELECT *`
    FROM Product
ORDER BY UnitPrice
```

Premise

```
(which Product
 sort :UnitPrice < )
```

Order by value descending

SQL

```
  SELECT *
    FROM Product
ORDER BY UnitPrice DESC
```

Premise

```
(which Product
 sort :UnitPrice > )
```

Limit number of results

SQL

```
  SELECT TOP 5 *
    FROM Product
ORDER BY UnitPrice
```

Premise

```
(top (which Product
      sort :UnitPrice < )
     5)
```

Paged results

SQL

```
   WITH part1 AS (
     SELECT *
     FROM Product
     ORDER BY UnitPrice)
   SELECT *
   FROM part1
   WHERE RRN(part1) BETWEEN 5 AND 10
```

Premise

```
(mid (which Product
      sort :UnitPrice < )
     5
     10)
```

Group by value

SQL

```
  SELECT TOP 100 UnitPrice
    FROM (SELECT Product.UnitPrice,
                 COUNT(*) AS Count
            FROM Product
        GROUP BY Product.UnitPrice)
  ORDER BY Count DESC
```

Premise

```
(with (top (with [Product as ?p :UnitPrice as ?u]
            list {:UnitPrice ?u
                  :Count (with [Product :UnitPrice as ?u] tally)}
            sort :Count > )
       100) each ?x
 list (:UnitPrice ?x))
```

Inner Join

SQL

```
    SELECT p.*
      FROM Product p,
           Category c
     WHERE c.CategoryName = "Beverages" AND
           c.CategoryID = p.CategoryID
```

Premise

```
(with
  [Product ^ as ?p :CategoryName = "Beverages" :CategoryId as ?id]
  [Category :Categoryid = ?id]
 list (premise ?p))
```

Left Join

SQL

```
SELECT c.CustomerID, COUNT(o.OrderID)
      FROM Customer c
 LEFT JOIN Order o
        ON o.CustomerID = c.CustomerID
  GROUP BY c.CustomerID
```

Premise

```
(with
  [Customer :CustomerId as ?i]
 list {:CustomerId ?i :Count (with [Order :CustomerId = ?i] tally)}
```

```
(with
  [Customer :CustomerId as ?i]
  [Order :OrderId as ?j]
  (left ?i ?j)
 group {?i (bracket (any ?i))}
 by 1 into ?group
 list {:CustomerId (@ ?group 1)
       :Count  (summation ?pair in (rest ?g) (@ ?pair 2)) }
```

Concatenate

SQL

```
SELECT customer.CompanyName
  FROM Customer AS customer
 WHERE customer.CompanyName LIKE "A%'
 UNION ALL
SELECT customer.CompanyName
  FROM Customer AS customer
 WHERE customer.CompanyName LIKE "E%'
```

Premise

```
(union
  (with [Customer :CompanyName as ?n] (like ?n "A%") list ?n)
  (with [Customer :CompanyName as ?n] (like ?n "E%") list ?n))
```

Create, Update and Delete

SQL

```
INSERT INTO Category (CategoryName, Description)
     VALUES ("Merchandising", "Cool products");

INSERT INTO Product (ProductName, CategoryID)
     SELECT TOP (1) "Red Jacket", CategoryID
       FROM Category
      WHERE CategoryName = "Merchandising";

UPDATE Product
   SET Product.ProductName = "Green Jacket"
 WHERE Product.ProductName = "Red Jacket";

DELETE FROM Product
 WHERE Product.ProductName = "Green Jacket";

DELETE FROM Category
 WHERE Category.CategoryName = "Merchandising";
```

Premise

```
(new Category :CategoryName "Merchandising"
              :Description "Cool Products")

(new Product :ProductName "Red Jacket"
             :CategoryId (which Category :CategoryName = Merchandising limit 1))

(with [Product ^ as ?p :ProductName = "Red Jacket"]
  do (put ?p :ProductName "Green Jacket"))

(with [Product ^ as ?p :ProductName = "Green Jacket"]
   do (old ?p))

(with [Category ^ as ?c :CategoryName = "Merchandising"]
   do (old ?c))
```

Recursive Query

SQL

```
WITH EmployeeHierarchy (EmployeeID,
                        LastName,
                        FirstName,
                        ReportsTo,
                        HierarchyLevel) AS
( SELECT EmployeeID
         , LastName
         , FirstName
         , ReportsTo
         , 1 as HierarchyLevel
      FROM Employees
     WHERE ReportsTo IS NULL

     UNION ALL

    SELECT e.EmployeeID
         , e.LastName
         , e.FirstName
         , e.ReportsTo
         , eh.HierarchyLevel + 1 AS HierarchyLevel
      FROM Employees e
INNER JOIN EmployeeHierarchy eh
        ON e.ReportsTo = eh.EmployeeID
) SELECT *
    FROM EmployeeHierarchy
ORDER BY HierarchyLevel, LastName, FirstName
```

Premise

```
  (relation EmployeeHierarchy
    :EmployeeId  :LastName  :FirstName :ReportsTo  :HierarchyLevel )

  (with (union
          (with [Employee ^ as ?e :ReportsTo is nothing :EmployeeId as ?i
                          :LastName as ?n :FirstName as ?f]
           list (knew EmployeeHierarchy :EmployeeId ?i :LastName ?n
                      :FirstName ?f :HierarchyLevel 1))
          (with
            [EmployeeHierarchy ^ as ?eh :EmployeeId as ?i :HierarchyLevel as ?h]
            [Employee ^ as ?e :EmployeeId as ?j :LastName as ?n :FirstName as ?f
                       :ReportsTo ?i]
           list (knew EmployeeHierarchy :EmployeeId ?j :LastName ?n
                      :FirstName ?f :HierarchyLevel (+ ?h 1))))
     as ?result
   list (premise ?result)
   sort :HierarchyLevel < :LastName < :FirstName < )
```

Bibliography

1. Blumberg , Micah & Miller, Michael S. P. 2025
   Building Sentient Beings, SubThought Corporation, DOI: 10.5281/zenodo.15522356

2. Chambers, C. 1992
   The Design and Implementation of the SELF Compiler, an Optimizing Compiler for Object-Oriented Programming Languages

3. Forgy, Charles 1979
   On the Efficient Implementation of Production Systems
   PhD Thesis, Department of Computer Science, Carnegie Mellon University, Pittsburgh PA

4. Forgy, Charles 1981
   OPS5 User's Manual, Technical Report CMU-CS-81-135 (Carnegie Mellon University)

5. Krishnamurthi, Sriram 2017
   Programming Languages: Application and Interpretation, second Edition

6. Linker, Sheldon O. 2005
   A knowledge base and question answering system based on LOGLAN and English

7. Linker, Sheldon O., Miller, M. S. P. 2014
   Association for the Advancement of Artificial Intelligence Spring Symposium Series (AAAI SSS 14)
   Implementing Selves with Safe Motivational Systems and Self-Improvement - Invited Talk
   "Premise: A Language for Cognitive Systems"

8. Miller, Michael S. P. 2013
   The Construction of Reality in a Cognitive System

9. Miller, Michael S. P. 2018
   Building Minds with Patterns, ISBN 978-0-692-54140-1

10. Queinnec, Christian 1994
    Lisp in Small Pieces, Cambridge University Press, ISBN 0-521-54566-8

11. Steele Jr , Guy L. 1981
    Common LISP: The Language, 2nd Ed., Digital Press, ISBN 978-1-55558-041-4

12. Stonebreaker, Michael 1993
    The Integration of Rule Systems and Database Systems
    Technical Report, University of California Berkeley

Index